

Universidad
Rey Juan Carlos

Escuela Técnica Superior
de Ingeniería Informática

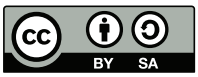
Grado en Ingeniería Informática

Curso 2025-2026

Trabajo Fin de Grado

SPIRITBLADE: APLICACIÓN WEB PARA ANÁLISIS DE DATOS DE LEAGUE OF LEGENDS

Autor: Jorge Andrés Echevarría
Tutor: Iván Chicano Capelo
Tutor: Francisco Gortázar Bellas



©2026 Jorge Andrés Echevarría
Algunos derechos reservados
Este documento se distribuye bajo la licencia
"Atribución-CompartirIgual 4.0 Internacional" de Creative Commons,
disponible en
https://creativecommons.org/licenses/by-sa/4.0/deed.es

Agradecimientos

A mi familia y amigos, por su comprensión, ánimo incondicional y por haberme acompañado a lo largo de todos mis años de estudio en la universidad.

Resumen

El presente Trabajo de Fin de Grado (TFG) describe el diseño y desarrollo de SPIRITBLADE, una aplicación web orientada al análisis estadístico avanzado de jugadores del videojuego competitivo *League of Legends*. El objetivo principal ha consistido en construir una plataforma integral que permita a los usuarios buscar y visualizar métricas de rendimiento en tiempo real, integrando la API oficial de Riot Games. Como valor añadido, el sistema incorpora el uso de Inteligencia Artificial (a través de la API de Google Gemini) para generar análisis personalizados sobre el historial de partidas, identificando fortalezas, debilidades y áreas de mejora táctica.

A nivel arquitectónico, el proyecto se ha implementado bajo el paradigma de *Single Page Application* (SPA, por sus siglas en inglés). El lado del servidor (backend) ha sido desarrollado empleando Java y el ecosistema Spring Boot, definiendo una robusta API REST. La arquitectura de persistencia se apoya en una base de datos relacional (MySQL) para la gestión de entidades y en un sistema de almacenamiento de objetos (*Object Storage*) basado en MinIO para la gestión de recursos multimedia. Asimismo, la seguridad queda garantizada mediante un sistema de autenticación y autorización basado en JSON Web Tokens (JWT, por sus siglas en inglés). Por su parte, la interfaz de cliente (frontend) ha sido construida con el *framework* Angular, ofreciendo una experiencia interactiva, modular y adaptativa.

Finalmente, el ciclo de vida del desarrollo ha seguido una metodología ágil e iterativa, aplicando prácticas de control de calidad mediante flujos de Integración Continua (CI, por sus siglas en inglés) en GitHub Actions. Esto incluye la ejecución automatizada de pruebas unitarias, de integración y *End-to-End* (E2E, por sus siglas en inglés), además de análisis estático de código con SonarCloud. La aplicación ha sido empaquetada mediante Docker y Docker Compose, facilitando su portabilidad y sentando las bases tecnológicas para un posterior despliegue automatizado en infraestructuras *Cloud*, aspecto que conformará el segundo TFG correspondiente a la doble titulación.

Palabras clave:

- League of Legends
- Java (Spring Boot)
- Angular
- Arquitectura Web
- Inteligencia Artificial
- Docker

Abstract

This Undergraduate Final Project describes the design and development of SPIRIT-BLADE, a web application dedicated to the advanced statistical analysis of players in the competitive video game *League of Legends*. The primary objective has been to build a comprehensive platform that allows users to search for and visualize real-time performance metrics retrieved from the official Riot Games API. As a core value proposition, the system integrates Artificial Intelligence (via the Google Gemini API) to generate personalized analysis of match history, identifying tactical strengths, weaknesses, and areas for improvement.

From an architectural standpoint, the project follows the Single Page Application (SPA) paradigm. The server-side (backend) was developed using Java and the Spring Boot ecosystem, defining a robust REST API. The persistence layer relies on a relational database (MySQL) for entity management and an Object Storage system based on MinIO for multimedia assets. Security is ensured through a stateless authentication and authorization system based on JSON Web Tokens (JWT). The client interface (frontend) was built with the Angular framework, providing an interactive, modular, and responsive experience.

The development lifecycle followed an agile and iterative methodology, applying quality assurance practices through Continuous Integration (CI) pipelines in GitHub Actions. This includes the automated execution of unit, integration, and End-to-End (E2E) tests, alongside static code analysis with SonarCloud. The application has been containerized using Docker and Docker Compose, ensuring portability and laying the technological foundation for future automated deployment in Cloud infrastructures, which will constitute the second part of this double-degree project.

Keywords:

- League of Legends
- Java (Spring Boot)
- Angular
- Web Architecture
- Artificial Intelligence
- Docker

Índice de contenidos

Índice de tablas

Índice de figuras

Índice de códigos

1. Introducción y objetivos **1**

1.1. Contexto . . . 1

1.2. Estado del arte . . . 2

1.2.1. Contexto académico: doble titulación . . . 3

1.3. Estructura de la memoria . . . 4

1.4. Objetivos . . . 5

2. Metodología **7**

3. Descripción funcional **9**

3.1. Descripción de la aplicación . . . 9

3.2. Funcionalidades y casos de uso . . . 10

3.2.1. Módulo de usuario no registrado . . . 10

3.2.2. Módulo de usuario registrado . . . 10

3.2.3. Módulo de administración . . . 11

3.3. Vídeo demostrativo . . . 11

4. Descripción informática **14**

4.1. Tecnologías . . . 15

4.1.1. Tecnologías de Backend . . . 15

4.1.2. Tecnologías de Frontend . . . 17

4.1.3. Almacenamiento y Persistencia de Datos . . . 18

4.1.4. Tecnologías de Seguridad Web . . . 18

4.1.5. APIs y Servicios de Terceros . . . 18

4.2. Herramientas . . . 19

4.2.1. Control de Versiones y Alojamiento . . . 19

4.2.2. Gestión de Dependencias y Construcción . . . 20
4.2.3. Entornos de Desarrollo Integrados (IDEs) . . . 20
4.2.4. Herramientas de Pruebas y Control de Calidad . . . 21
4.2.5. Automatización: Integración Continua (CI) . . . 22
4.2.6. Virtualización para Desarrollo Local . . . 24
4.3. Arquitectura . . . 24
4.3.1. Arquitectura del Servidor (Backend) . . . 24
4.3.2. Patrones de Diseño: DTO y Mappers . . . 26
4.3.3. Arquitectura del Cliente (Frontend) . . . 27
4.3.4. Consideraciones Arquitectónicas y Riesgos (Dependencia de Terceros) . . . 27
4.3.5. Modelo de dominio . . . 28
4.3.6. Entidad `UserModel` . . . 28
4.3.7. Consideraciones de Rendimiento: Desacoplamiento del Catálogo de Campeones . . . 29
4.3.8. Entidad `Summoner` . . . 29
4.3.9. Entidad `MatchEntity` . . . 30
4.3.10. Entidad `RankHistory` . . . 30
4.3.11. Entidad `Champion` . . . 31
4.4. Implementación . . . 33
4.4.1. Integración con Riot Data Dragon . . . 33
4.4.2. Arquitectura de Seguridad y Gestión de Errores . . . 33
4.4.3. Algoritmo de Reconstrucción de LP y Sincronización Diferencial . . . 33
4.4.4. Algoritmo de Procesamiento y Análisis mediante IA . . . 35
4.5. Control de calidad . . . 36
4.5.1. Estrategia de Aseguramiento de la Calidad: La Pirámide de Pruebas 36
4.5.2. Pruebas Unitarias y el Desafío de la Simulación (*Mocking*) . . . 37
4.5.3. Análisis estático de código . . . 38
4.6. Empaquetado y distribución . . . 38
4.7. Proceso de desarrollo . . . 39
4.7.1. Fase 1: Análisis de Requisitos y Diseño de Interfaces . . . 40
4.7.2. Fase 2: Infraestructura Base, Repositorio y Pruebas . . . 40
4.7.3. Fase 3: Funcionalidad básica y Docker . . . 40
4.7.4. Fase 4: Funcionalidad intermedia y avanzada . . . 41
4.7.5. Fase 5: Escritura de la memoria . . . 41
4.7.6. Fase 6: Preparación de la presentación . . . 41
4.7.7. Gestión de tareas . . . 42

4.7.8. Git y Estrategia de Ramas . . . 43
4.7.9. Integración y entrega continua . . . 43
4.7.10. Versionado . . . 44

5. Conclusiones y trabajos futuros **47**
5.1. Conclusiones . . . 47
5.1.1. Consecución de objetivos funcionales y técnicos . . . 47
5.1.2. Reflexión y lecciones aprendidas . . . 48
5.1.3. Conclusiones personales . . . 49
5.2. Trabajos futuros . . . 50

Bibliografía **52**

Apéndices **54**

A. Funcionalidades detalladas **56**
A.1. Módulo de usuario no registrado . . . 56
A.2. Módulo de usuario registrado . . . 59
A.3. Módulo de administración . . . 65

B. Instrucciones para ejecutar la aplicación **67**
B.1. Requisitos previos . . . 67
B.2. Obtención del código fuente . . . 67
B.3. Configuración del entorno (.env) . . . 68
B.4. Ejecución del sistema . . . 68
B.5. Acceso a la aplicación y credenciales . . . 68

C. Ejecución y edición de código **70**
C.1. Requisitos técnicos . . . 70
C.2. Obtención del código fuente . . . 71
C.3. Lanzamiento de los módulos en desarrollo . . . 71
C.3.1. Servidor (Backend) . . . 71
C.3.2. Cliente (Frontend) . . . 71
C.4. Compilación para producción (build) . . . 72

Índice de tablas

2.1. Cronograma de hitos del proyecto (fases funcionales). 8

Índice de figuras

2.1. Diagrama de Gantt ilustrando la planificación del proyecto. 8

3.1. Perfil de un invocador (*summoner*) visto por un usuario no registrado. . 10
3.2. Panel de control principal del usuario registrado. 12
3.3. Vista de un invocador con el análisis de rendimiento generado por IA. . . 13
3.4. Panel de administración para la gestión de usuarios y monitorización del sistema. 13

4.1. Interfaz interactiva de Swagger documentando la API REST de SPIRIT-BLADE. Se observa el catálogo general de controladores y la correcta distinción entre rutas públicas y protegidas mediante autorización JWT (indicadas con un candado). 22
4.2. Detalle de un *endpoint* específico en Swagger UI, mostrando los parámetros requeridos, el esquema de respuesta JSON (DTO) y el botón de autorización. 23
4.3. Diagrama de arquitectura y flujo de comunicación entre la SPA y el ecosistema de APIs externas. 25
4.4. Diagrama de clases del dominio: Relación entre el Usuario (`UserModel`) y el Invocador (`Summoner`). 31
4.5. Diagrama de clases del dominio: Catálogo de Campeones (`Champion`) desacoplado del historial. 31
4.6. Diagrama de clases del dominio: Relación del Invocador con el historial de partidas (`MatchEntity`) y la progresión de rango (`RankHistory`). . . . 32
4.7. Panel de control de SonarCloud mostrando el estado satisfactorio del *Quality Gate* y métricas del tamaño del código. 38
4.8. Vista detallada del *Backlog* del proyecto en GitHub Projects. 42
4.9. Representación visual del árbol de ramas siguiendo la metodología de features. 43
4.10. Ejemplo de revisión de código automatizada en los PR. 44
4.11. Ejemplo de sugerencias de mejora en un PR. 45

A.1. Búsqueda de invocadores en la página principal. 57
A.2. Información general del perfil y rango de un invocador. 57
A.3. Estadísticas de campeones más jugados. 57
A.4. Listado de búsquedas recientes de invocadores. 58
A.5. Detalles específicos de una partida en el historial. 58
A.6. Historial de partidas con paginación integrada. 58
A.7. Panel de control del usuario registrado. 60
A.8. Interfaz para enlazar la cuenta de Riot Games. 61
A.9. Gestión de la foto de perfil del usuario. 61
A.10.Formulario de edición de datos de perfil. 62
A.11.Pantalla de inicio de sesión y registro en la plataforma. 62
A.12.Estadísticas clave del invocador vinculado al usuario. 63
A.13.Sección de invocadores favoritos para acceso rápido. 63
A.14.Análisis de rendimiento generado por Inteligencia Artificial. 63
A.15.Gráfico de evolución del rango (LP) a lo largo del tiempo. 64
A.16.Vista general del panel de administración. 65
A.17.Tabla interactiva con el listado de usuarios del sistema. 65
A.18.Filtros de búsqueda avanzada en el panel de administración. 66

Índice de códigos

4.1. Algoritmo de iteración y cálculo inverso de Puntos de Liga (LP). 34
4.2. Construcción del contexto y el System Prompt para la API de Gemini. . 35

1 Introducción y objetivos

Este capítulo establece el marco inicial del proyecto, describiendo el contexto de los deportes electrónicos y el estado actual de las herramientas de análisis en *League of Legends*. Asimismo, se detallan la motivación del proyecto y la estructura general de la presente memoria, introduciendo el desarrollo a través de un proyecto de doble titulación.

1.1. Contexto

En la última década, los deportes electrónicos, comúnmente conocidos como *e-sports*, han experimentado un crecimiento exponencial, consolidándose como una de las industrias de entretenimiento digital más relevantes a nivel mundial. Dentro de este ecosistema competitivo, *League of Legends* (desarrollado por Riot Games) destaca como uno de los títulos más influyentes y jugados, reuniendo a millones de usuarios diarios y sosteniendo una infraestructura competitiva profesional que abarca ligas regionales y campeonatos mundiales.

La naturaleza intrínseca de *League of Legends* se basa en la toma de decisiones estratégicas, la gestión de recursos, el trabajo en equipo y la ejecución mecánica. A lo largo de una partida, se generan enormes cantidades de datos e indicadores de rendimiento que reflejan el comportamiento y la eficacia de cada jugador. En este escenario, el análisis de datos se ha convertido en una herramienta fundamental no solo para los equipos profesionales y sus cuerpos técnicos, sino también para los jugadores aficionados y competitivos que buscan optimizar su rendimiento en las colas clasificatorias (*Rankeds*).

Para satisfacer esta necesidad de información, Riot Games proporciona una Interfaz de Programación de Aplicaciones (API, del inglés *Application Programming Interface*) oficial que permite a los desarrolladores acceder a una vasta cantidad de datos en tiempo real sobre los invocadores (jugadores), historiales de partidas, estadísticas de campeones (personajes jugables del videojuego) y métricas de juego. Gracias a esta apertura de datos, ha surgido un sector dedicado a la creación de plataformas analíticas de terceros.

1.2. Estado del arte

En la actualidad, existen diversas herramientas y aplicaciones web muy populares en la comunidad de *League of Legends*, tales como *Porofessor*, *Blitz* o *OP.GG*. Estas plataformas han estandarizado la forma en la que los jugadores consultan sus estadísticas, ofreciendo resúmenes de rendimiento, tasas de victoria (*winrates*), promedios de Asesinatos, Muertes y Asistencias (KDA, del inglés *Kills, Deaths, Assists*), y sugerencias de equipamiento en tiempo real.

No obstante, a pesar de la utilidad indiscutible de estas herramientas, muchas de ellas tienden a ofrecer una experiencia homogeneizada y, a menudo, polarizada. Con frecuencia, se limitan a la presentación de datos en crudo o indicadores estadísticos clásicos o, por el contrario, saturan al usuario con un exceso abrumador de información, provocando que los datos verdaderamente relevantes queden sepultados y pasen desapercibidos. El proyecto SPIRITBLADE nace con la motivación de resolver esta brecha ofreciendo una plataforma de análisis de jugadores que, si bien se inspira en las herramientas líderes del sector, presenta una identidad visual, una marca y un enfoque completamente propios y centrados en aportar valor real al usuario.

El objetivo principal que justifica el desarrollo de SPIRITBLADE es proporcionar una experiencia de usuario orientada al análisis ágil y directo, dirigida específicamente a jugadores competitivos. El elemento diferenciador de esta plataforma radica en la interpretación de los datos mediante inteligencia artificial y en su presentación visualmente optimizada. Aunque las métricas clásicas, como el KDA o el *winrate*, son fundamentales, a menudo se muestran de forma descontextualizada. Por ello, SPIRITBLADE busca ir un paso más allá de la mera extracción y volcado masivo de datos, utilizando Inteligencia Artificial (IA) para analizar estas estadísticas y extraer conclusiones precisas. El propósito es filtrar el ruido analítico e ir directamente al grano, mostrando la información más relevante de manera sencilla y rápida de asimilar en un cuadro de mandos (*dashboard*) intuitivo. De esta forma, no se busca saturar al usuario con nuevas métricas, sino innovar en la lectura de las ya existentes para ayudarle a identificar sus áreas de mejora de un solo vistazo.

SPIRITBLADE se materializa como una aplicación web diseñada para ofrecer un rendimiento fluido y una interfaz de usuario moderna e inmersiva. El núcleo del *backend* de la aplicación se encarga de la orquestación y conexión con la API de Riot Games para extraer y procesar la información de los jugadores antes de servirla al cliente.

Entre las características fundamentales que se abordan en el desarrollo de la aplicación, destacan:

- **Búsqueda y perfil de usuario:** Un sistema que permite localizar a cualquier jugador mediante su *Riot ID*, mostrando de forma clara su rango actual, nivel e icono de invocador.
- **Análisis de historial de partidas:** Visualización estructurada de las partidas recientes (*Recent Matches*), identificando victorias y derrotas, duración de los encuentros, campeones utilizados y las métricas individuales de KDA.
- **Estadísticas de campeones:** Módulo dedicado a mostrar el desempeño histórico del usuario con sus campeones más jugados, calculando el número de partidas y el porcentaje de victorias asociado.
- **Análisis de rendimiento con Inteligencia Artificial:** Integración de un sistema automatizado que evalúa e interpreta las estadísticas clásicas del jugador, permitiendo al usuario identificar sus fortalezas y debilidades mediante lenguaje natural.
- **Identidad visual y experiencia de usuario:** Diseño de una interfaz responsiva y altamente tematizada que alinea la experiencia de navegación web con el contexto estético y competitivo del propio videojuego.

El desarrollo se ha llevado a cabo siguiendo metodologías rigurosas de ingeniería del *software* para asegurar la calidad, mantenibilidad y escalabilidad del código, implementando arquitecturas limpias, mejores prácticas y patrones de diseño tanto en el lado del cliente (*frontend*) como en el servidor (*backend*).

1.2.1. Contexto académico: doble titulación

La naturaleza de este documento se enmarca dentro del contexto académico de la Universidad Rey Juan Carlos (URJC). El autor del presente proyecto cursa la doble titulación de Grado en Ingeniería Informática (GII) y Grado en Ingeniería del Software (GIS). Para el desarrollo del presente TFG, se ha acordado la realización de dos proyectos diferenciados que se complementen entre ellos.

Por consiguiente, el proyecto global de SPIRITBLADE se divide en dos partes claramente delimitadas:

1. **Primer TFG (Objeto de esta memoria):** Se centra en la ingeniería de requisitos, el diseño y el desarrollo integral de la lógica de negocio y la interfaz de la aplicación web. Abarca la construcción del *frontend* y el *backend*, la integración con las APIs externas (Riot Games y servicios de IA generativa), el sistema de control de acceso y la persistencia de datos. Además, incluye la implementación de pruebas automáticas (unitarias, de integración y extremo a extremo (E2E, del inglés *End-to-End*)) y el análisis estático del código, sentando las bases de la integración continua (CI, del inglés *Continuous Integration*) para garantizar su calidad.
2. **Segundo TFG:** Constituye un trabajo posterior e independiente orientado a la arquitectura de sistemas, la escalabilidad y el despliegue. En dicho TFG se abordará la evolución de la arquitectura mediante la integración de nuevos servicios. Asimismo, contemplará la transición desde entornos de desarrollo locales hacia un despliegue *cloud* completo, incorporando automatización, despliegue continuo (CD, del inglés *Continuous Deployment*) y la realización de pruebas de carga sobre la infraestructura ampliada.

Cualquier mención en este documento a infraestructuras en la nube, contenedores o despliegues automatizados se realiza meramente a nivel contextual o como preparación arquitectónica, ya que la ejecución, configuración y evaluación de dichas competencias corresponden de forma exclusiva a la memoria del segundo TFG.

1.3. Estructura de la memoria

Para facilitar la comprensión y lectura del trabajo realizado, la presente memoria se ha estructurado en los siguientes capítulos:

- **Capítulo 1. Introducción y objetivos:** Este capítulo presenta el contexto del proyecto, el estado del arte, la motivación que lo impulsa, y detalla los objetivos funcionales y técnicos que se pretenden alcanzar.
- **Capítulo 2. Metodología:** Se describe en alto nivel cómo se ha desarrollado el trabajo, detallando las fases del ciclo de vida y la planificación temporal.
- **Capítulo 3. Descripción funcional:** Se enumeran las funcionalidades principales de la aplicación desarrollada.

- **Capítulo 4. Descripción informática:** Constituye el núcleo técnico de la memoria. Se documentan las tecnologías y herramientas utilizadas, la arquitectura de despliegue, el modelo de dominio, la API REST, los detalles de implementación de algoritmos clave, y las estrategias de control de calidad.
- **Capítulo 5. Conclusiones y trabajos futuros:** Se expone una reflexión final sobre el grado de cumplimiento de los objetivos y las posibles líneas de evolución funcional de la plataforma.

nditemize

1.4. Objetivos

Como conclusión a la problemática y motivación expuestas en las secciones anteriores, a continuación se detallan los objetivos concretos que se pretenden alcanzar con el desarrollo del proyecto SPIRITBLADE.

Es fundamental recordar que, debido a la naturaleza de la doble titulación (Grado en Ingeniería Informática y Grado en Ingeniería del Software), el ciclo de vida completo de esta plataforma se ha dividido en dos Trabajos de Fin de Grado. Por consiguiente, los objetivos aquí descritos se circunscriben de forma exclusiva al primer trabajo.

El objetivo principal de este proyecto es el diseño y desarrollo integral de una aplicación web *full-stack* orientada al análisis avanzado de datos del videojuego *League of Legends*. Se pretende construir una plataforma funcional, segura y con identidad visual propia que permita a los usuarios consultar estadísticas en tiempo real extraídas de la API oficial de Riot Games, aportando un valor añadido mediante la integración de IA para la evaluación del rendimiento de los jugadores.

Para lograr el objetivo general propuesto, el proyecto se ha desglosado en una serie de objetivos específicos, divididos en dos categorías: funcionales y técnicos.

Desde la perspectiva del usuario y la lógica de negocio (objetivos funcionales), se plantean las siguientes metas:

- **Módulo de acceso público:** Implementar un motor de búsqueda que permita a cualquier usuario, sin necesidad de registro previo, consultar el perfil de un jugador mediante su identificador oficial (*Riot ID*). Este perfil debe mostrar de forma dinámica su rango, nivel, historial de partidas recientes detallado (KDA, duración, resultado) y las estadísticas de maestría de sus campeones más utilizados.

- **Módulo de usuarios registrados:** Desarrollar un sistema de cuentas que proporcione un panel de control (*dashboard*) privado. El sistema debe permitir al usuario vincular su cuenta del videojuego, mantener una lista de jugadores favoritos para su seguimiento rápido y personalizar su perfil de usuario.
- **Análisis automatizado mediante IA:** Integrar un servicio de Inteligencia Artificial (haciendo uso de la API de Google Gemini) capaz de procesar el historial de partidas clasificatorias del usuario vinculado y generar un informe analítico sobre su desempeño, destacando fortalezas y áreas de mejora de manera personalizada.
- **Módulo de administración:** Construir un panel de control con acceso restringido para los administradores de la plataforma, que permita gestionar la base de datos de usuarios (búsqueda, bloqueo, eliminación y edición de perfiles).

Desde el punto de vista de la ingeniería del *software* (objetivos técnicos), se establecen los siguientes requerimientos para garantizar la robustez y mantenibilidad del sistema:

- **Desarrollo full-stack:** Construir una arquitectura SPA utilizando el *framework* Angular para el *frontend*, apoyada sobre una API REST desarrollada con Spring Boot y Java para el *backend*.
- **Seguridad web:** Implementar un sistema de autenticación seguro y sin estado (*stateless*) basado en *JSON Web Tokens* (JWT) para proteger los *endpoints* privados y asegurar la integridad de las sesiones de usuario.
- **Aseguramiento de la Calidad (QA, del inglés *Quality Assurance*):** Diseñar e implementar una batería de pruebas automáticas que abarque pruebas unitarias, pruebas de integración y pruebas E2E, garantizando el correcto funcionamiento de los casos de uso críticos.
- **Análisis estático de código:** Integrar herramientas de revisión continua de código para detectar vulnerabilidades, reducir la deuda técnica y asegurar el cumplimiento de los estándares de programación y las mejores prácticas de la industria.

2

Metodología

Este capítulo describe el enfoque de trabajo empleado para la realización del proyecto, detallando las fases del ciclo de vida del *software* y la planificación temporal a través de un cronograma y un diagrama de Gantt.

El desarrollo de este TFG se ha estructurado siguiendo un proceso iterativo e incremental, planificado para cumplir con los hitos académicos y garantizar un progreso constante. A continuación se detallan las fases en las que se ha dividido el trabajo, indicando en qué sección del documento se profundiza sobre cada aspecto:

- **Fase 1: Definición de funcionalidades.** Durante esta fase inicial se analizaron los requisitos del sistema y se diseñó la interfaz de usuario. Las funcionalidades generales y detalladas concebidas en esta etapa se describen exhaustivamente en el **Capítulo 3**.
- **Fase 2: Configuración de las tecnologías y herramientas de desarrollo.** En esta etapa se establecieron las bases tecnológicas del proyecto y se configuraron los controles de calidad periódicos. La justificación de las tecnologías seleccionadas y las herramientas utilizadas se detalla en el **Capítulo 4**, mientras que los mecanismos de control de calidad se explican en sus respectivas secciones de aseguramiento de la calidad.

- **Fases 3 y 4: Desarrollo iterativo e incremental.** El núcleo del proyecto consistió en la construcción progresiva de la aplicación mediante la publicación de versiones funcionales (*releases*). El proceso de desarrollo técnico empleado, así como la arquitectura y algoritmos resultantes, se describen en detalle a lo largo de todo el **Capítulo** 4.

- **Fase 5: Redacción de memoria.** Documentación formal del proyecto, incluyendo el detalle de la arquitectura y la implementación técnica, desarrollada durante la pausa temporal (marzo-abril de 2026) tras la finalización del desarrollo de la aplicación y en paralelo a la pausa del segundo proyecto.

- **Fase 6: Defensa.** Preparación de las diapositivas y exposición ante el tribunal evaluador.

Para ilustrar la planificación y ejecución de las fases descritas, se ha elaborado un cronograma con las fechas de inicio y fin estimadas para cada etapa principal. En la Tabla 2.1 se presenta el resumen de fechas:

Fase	**Inicio**	**Fin**
Fase 1: Definición de funcionalidades	01/09/2025	15/09/2025
Fase 2: Configuración de tecnologías	16/09/2025	01/10/2025
Fases 3 y 4: Desarrollo iterativo	02/10/2025	01/12/2025
Fase 5: Redacción de memorias	16/03/2026	25/05/2026
Fase 6: Preparación y defensa	26/05/2026	09/06/2026

Tabla 2.1: Cronograma de hitos del proyecto (fases funcionales).

De forma complementaria, en la Figura 2.1 se ilustra el diagrama de Gantt correspondiente al ciclo de vida del proyecto, ofreciendo una representación gráfica temporal de las fases.

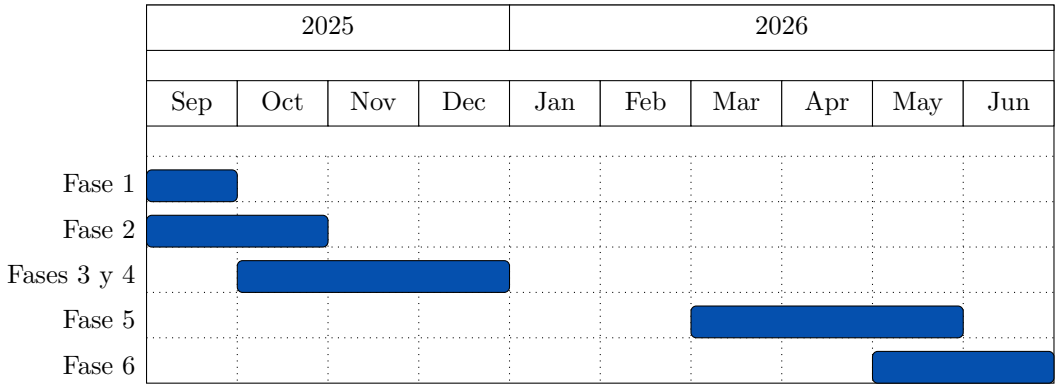


Figura 2.1: Diagrama de Gantt ilustrando la planificación del proyecto.

3

Descripción funcional

Este capítulo describe la aplicación SPIRITBLADE desde una perspectiva funcional, detallando qué necesidades resuelve para el usuario final y cómo se estructura su flujo de navegación a través de sus funcionalidades.

3.1. Descripción de la aplicación

SPIRITBLADE es una plataforma de análisis estadístico para el videojuego *League of Legends*. Su propósito es transformar los datos masivos y complejos proporcionados por la API oficial de Riot Games en información útil y fácil de asimilar mediante el uso de Inteligencia Artificial. La aplicación permite a los usuarios buscar perfiles de jugadores, que en adelante denominaremos invocadores o *summoners*, analizar sus partidas recientes y obtener un informe personalizado sobre su estilo de juego, destacando errores tácticos y áreas de mejora.

3.2. Funcionalidades y casos de uso

A continuación, se resumen las funcionalidades principales del sistema divididas según el perfil del usuario (Anónimo, Registrado o Administrador). En cada sección se incluyen capturas de pantalla de la interfaz correspondientes, ilustrando la línea estética adoptada.

Se indicará que el **listado detallado de funcionalidades** (junto con la explicación técnica pormenorizada de cada una) se encuentra referenciado y extendido en el **Anexo A**.

3.2.1. Módulo de usuario no registrado

El alcance para el usuario no registrado se centra en la consulta ágil de perfiles y estadísticas generales del videojuego (como se aprecia en la Figura 3.1). Permite, de manera abierta, buscar invocadores por su *Riot ID*, visualizar su rango competitivo actual, consultar sus campeones más jugados e inspeccionar el historial paginado de sus partidas recientes.

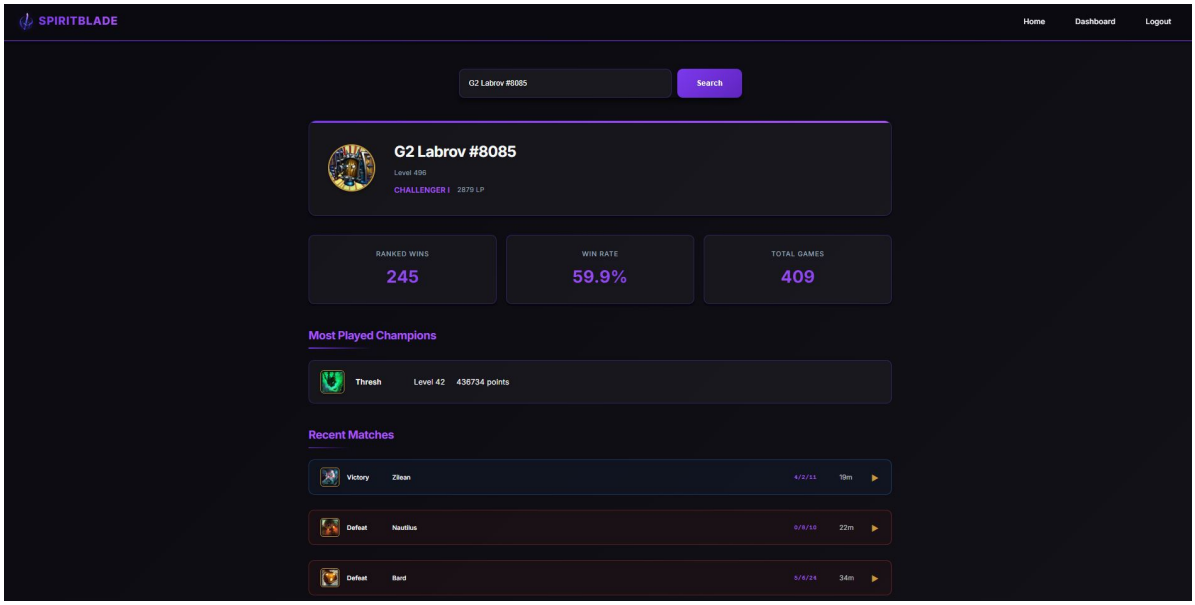


Figura 3.1: Perfil de un invocador (*summoner*) visto por un usuario no registrado.

3.2.2. Módulo de usuario registrado

Los usuarios autenticados acceden a capacidades de personalización y análisis avanzado mediante un panel de control privado, ilustrado en las Figuras 3.2 y 3.3. Mediante

un proceso de registro y *login*, estos usuarios pueden vincular su cuenta oficial de Riot, gestionar un avatar personalizado, mantener listas de invocadores favoritos y, de manera fundamental, ejecutar análisis tácticos impulsados por Inteligencia Artificial sobre su rendimiento reciente.

3.2.3. Módulo de administración

Capacidades exclusivas para la gestión de la comunidad y moderación (Figura 3.4). Los administradores cuentan con un panel de control interactivo donde pueden buscar usuarios avanzados, visualizar el estado del sistema y habilitar, deshabilitar o eliminar permanentemente las cuentas que infrinjan las normas de uso.

3.3. Vídeo demostrativo

Para facilitar la comprensión del funcionamiento de la interfaz y la fluidez de la aplicación, se ha grabado una demostración de uso. El vídeo está disponible en el siguiente enlace: `https://youtu.be/JmKdQhNpecU`.

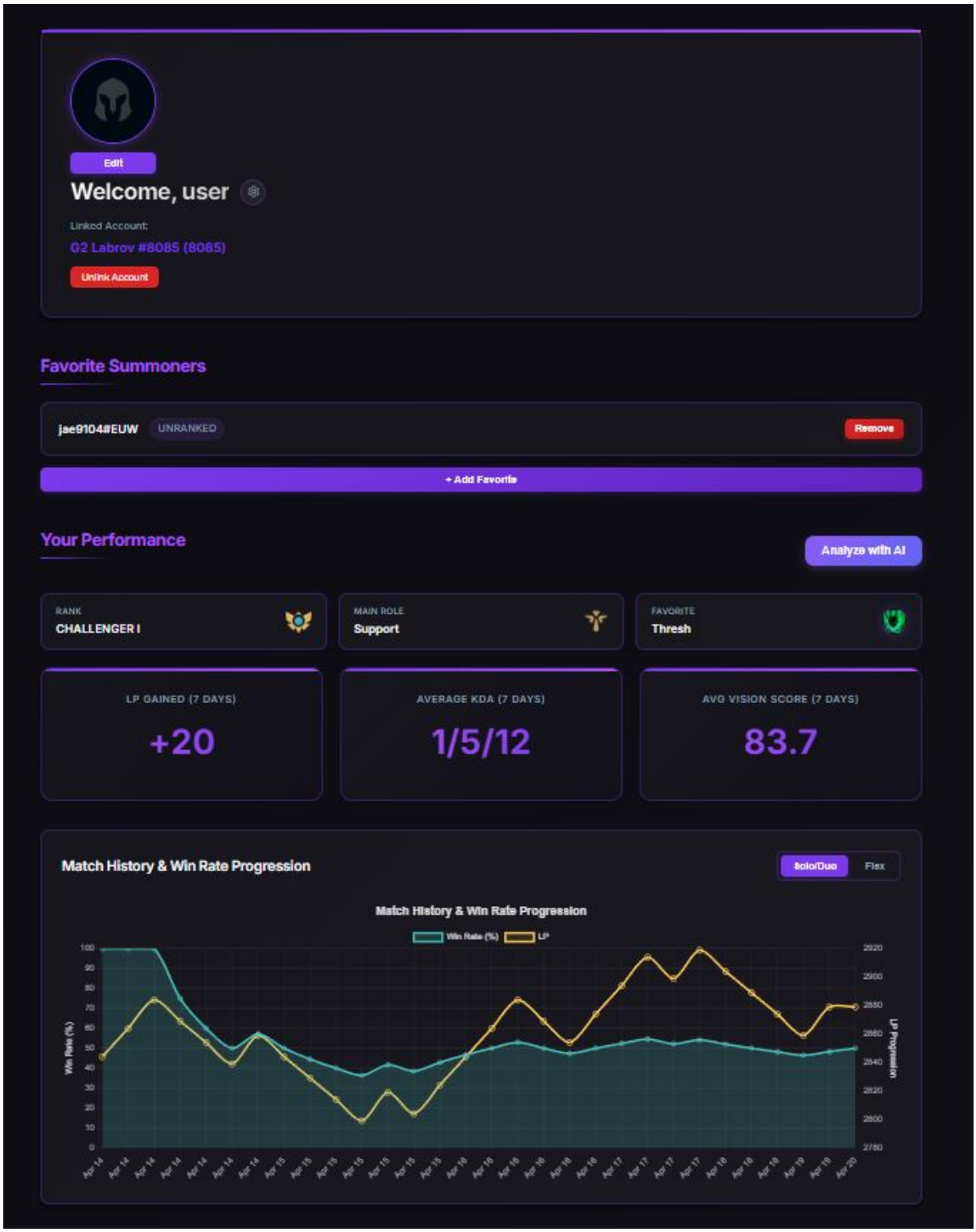


Figura 3.2: Panel de control principal del usuario registrado.

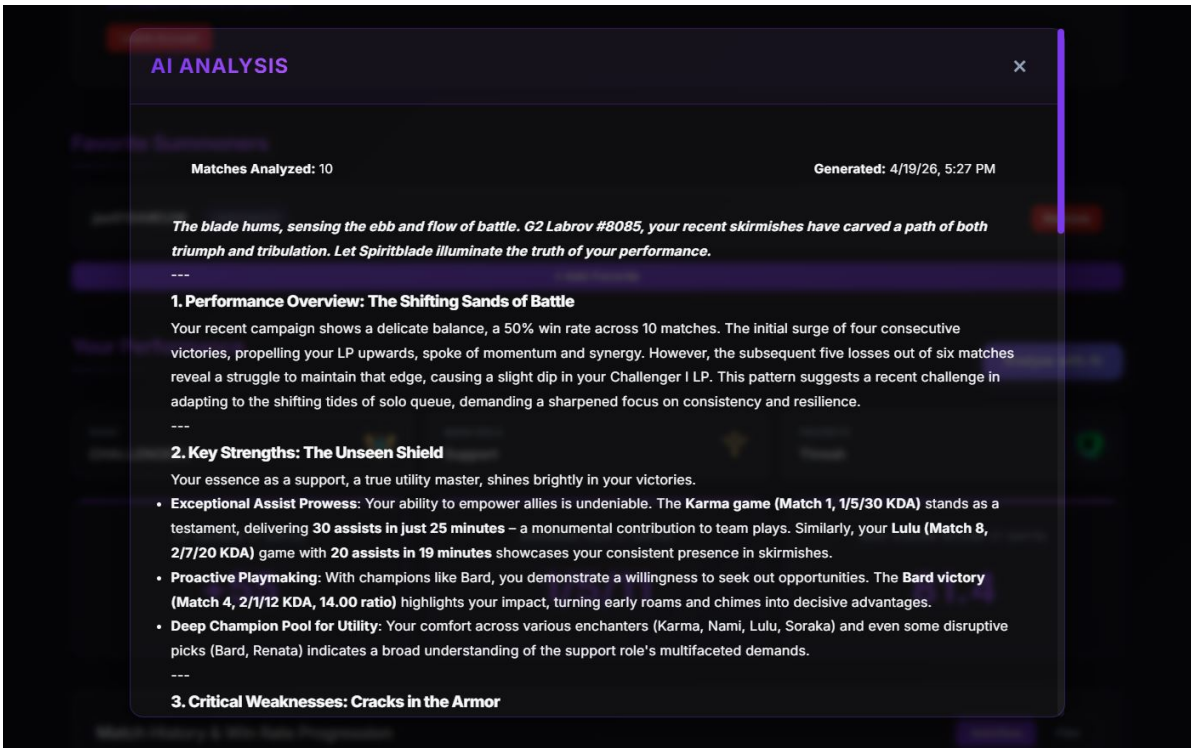


Figura 3.3: Vista de un invocador con el análisis de rendimiento generado por IA.

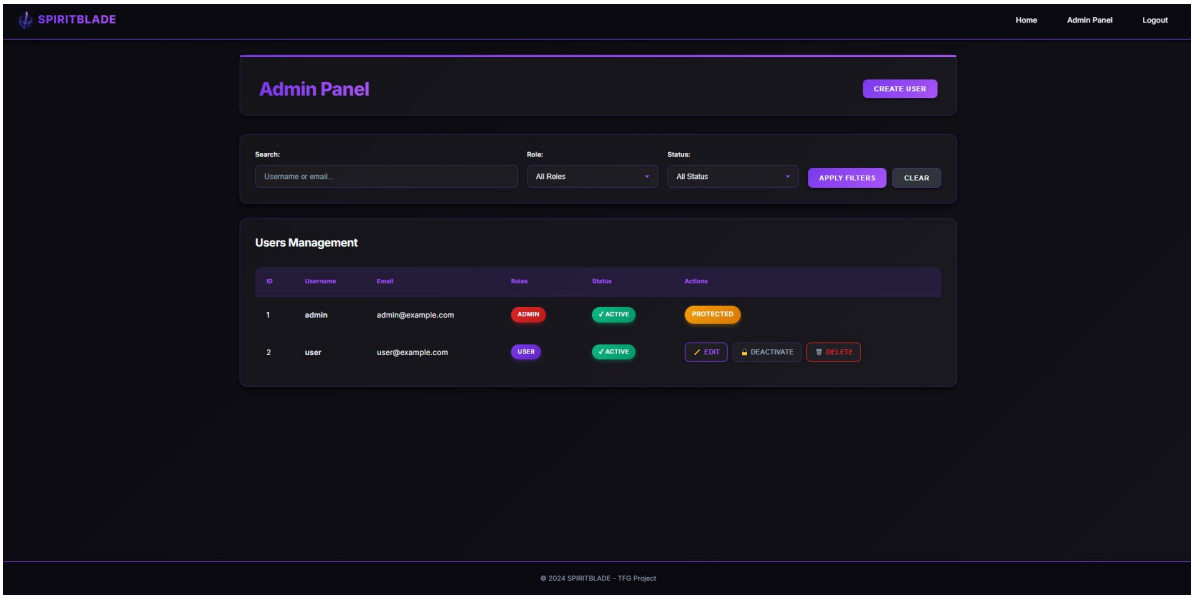


Figura 3.4: Panel de administración para la gestión de usuarios y monitorización del sistema.

4

Descripción informática

Este capítulo recoge todos los aspectos informáticos de la aplicación. El código fuente de la plataforma SPIRITBLADE se encuentra disponible en el repositorio de GitHub: `https://github.com/codeurjc-students/2025-SPIRITBLADE`.

La aplicación web sigue una arquitectura SPA, dividida en un cliente, un servidor y una base de datos relacional. A continuación, se presenta un resumen de la arquitectura de despliegue:

- **Tipo:** Web SPA, API REST.
- **Tecnologías:** Java 21, Spring Boot 3, Angular 17, MySQL, JWT.
- **Herramientas:** Docker, Git, Maven, npm, Swagger, SonarCloud.
- **Control de calidad:** Pruebas unitarias (JUnit, Mockito), Integración continua (GitHub Actions), Análisis estático (SonarCloud).
- **Despliegue:** Contenerización local mediante Docker y Docker Compose.
- **Proceso de desarrollo:** Metodología ágil, iterativa e incremental, siguiendo GitHub Flow.

4.1. Tecnologías

La arquitectura de SPIRITBLADE se sustenta en un modelo cliente-servidor, desacoplando completamente la lógica de presentación (*Frontend*) de la lógica de negocio y el acceso a los datos (*Backend*). Esta separación favorece el mantenimiento independiente de ambas capas y permite una futura escalabilidad. A continuación, se describen las tecnologías empleadas en cada uno de estos ámbitos.

4.1.1. Tecnologías de Backend

El servidor de la aplicación, o *Backend*, constituye el núcleo computacional de SPIRITBLADE. Es el responsable de orquestar la comunicación con servicios externos, interactuar con la base de datos persistente, gestionar la seguridad, verificar la identidad de los usuarios y proporcionar los datos procesados al cliente a través de una API REST.

Java

Se ha escogido Java [1] como el lenguaje de programación base para el desarrollo del servidor, concretamente en su versión 21. Java es un lenguaje orientado a objetos, fuertemente tipado, compilado e interpretado (a través de la Máquina Virtual de Java o JVM), que goza de un amplio respaldo en el sector del desarrollo de *software* empresarial debido a su robustez, portabilidad e inmenso ecosistema de librerías.

La elección específica de la versión 21, al ser una versión de Soporte a Largo Plazo (LTS, del inglés *Long Term Support*), garantiza estabilidad y actualizaciones de seguridad continuas. Además, incorpora características modernas que mejoran la eficiencia en el procesamiento concurrente y simplifican la sintaxis general del código, facilitando el desarrollo de la lógica de negocio compleja que requiere el procesamiento masivo de datos provenientes del videojuego.

Spring Boot y Spring Framework

Para la construcción de la API REST, se ha optado por el uso de Spring Boot 3 [2]. Spring Boot es un subproyecto dentro del vasto ecosistema de Spring Framework que aplica el paradigma de "convención sobre configuración". Esta filosofía permite levantar una aplicación web completa y autoejecutable (con un servidor Apache Tomcat embebido) en un tiempo notablemente reducido, minimizando el código repetitivo (*boilerplate*).

Dentro del marco de trabajo de Spring, el proyecto SPIRITBLADE hace uso extensivo de varios de sus módulos principales:

- **Spring Web:** Módulo fundamental encargado de proporcionar las capacidades necesarias para construir la arquitectura RESTful. Gestiona el enrutamiento de las peticiones HTTP entrantes hacia los controladores correspondientes (tales como `UserController`, `SummonerController` o `DashboardController`), administrando los códigos de estado HTTP y facilitando la serialización y deserialización de objetos de transferencia de datos (DTOs, del inglés extitData Transfer Objects) a formato JSON mediante la librería Jackson.
- **Spring Data JPA:** Constituye la capa de persistencia y abstracción de la base de datos. Hace uso de la especificación *Java Persistence API* (JPA) y utiliza Hibernate como proveedor de *mapping* objeto-relacional (ORM, del inglés *Object-Relational Mapping*). Esto ha permitido definir entidades de dominio directamente en código Java e interactuar con la base de datos relacional sin necesidad de redactar sentencias SQL nativas de forma constante, lo cual agiliza el desarrollo y mitiga riesgos de seguridad como la inyección SQL.
- **Spring Security:** Se ha empleado este módulo especializado para implementar un robusto control de acceso y autenticación en la plataforma. Configura los filtros de intercepción de tráfico, asegurando que los *endpoints* privados (como el panel de administración `AdminController`) estén protegidos y requieran autorización explícita antes de ejecutar cualquier lógica de negocio.

Project Lombok

En el desarrollo tradicional con Java, la verbosidad del código —especialmente la necesidad de escribir explícitamente métodos de acceso, mutadores y constructores— puede dificultar la lectura y el mantenimiento del proyecto. Para mitigar este problema de sobreabundancia de código repetitivo (*boilerplate*), se ha integrado la librería **Project Lombok** [3] en la capa del servidor.

Lombok actúa directamente en la fase de compilación, inyectando automáticamente el código estructural necesario mediante el uso de anotaciones declarativas (tales como `@Getter`, `@Setter`, `@NoArgsConstructor` o `@Builder`). Su aplicación intensiva en las entidades del modelo de dominio y en los objetos de transferencia de SPIRITBLADE ha reducido de forma drástica el volumen de líneas de código fuente. Esta decisión técnica no solo agiliza el desarrollo, sino que mejora significativamente la legibilidad del repositorio, permitiendo focalizar la atención estrictamente en la lógica de negocio.

4.1.2. Tecnologías de Frontend

El cliente de la aplicación web se ha diseñado bajo el paradigma de una SPA. Esto significa que la aplicación carga un único documento HTML y reescribe dinámicamente su contenido interactuando con las APIs, proporcionando una experiencia de usuario fluida, rápida y sin recargas de página intermedias.

TypeScript

Como lenguaje de programación para el lado del cliente, se ha utilizado TypeScript [4], un superconjunto desarrollado por Microsoft que añade un sistema de tipos estático y programación orientada a objetos estricta sobre JavaScript. El uso de TypeScript resulta fundamental en proyectos de esta magnitud para detectar errores semánticos en tiempo de compilación, mejorar el autocompletado en los entornos de desarrollo integrados (IDE, del inglés extitIntegrated Development Environment) y facilitar la comprensión de las estructuras de datos devueltas por el *backend*.

Angular

El *framework* seleccionado para la construcción y estructuración de la interfaz gráfica ha sido Angular (en su versión 17) [5], mantenido por Google. A diferencia de otras librerías como React, Angular provee un ecosistema cerrado y completo (*framework* opinado) que incluye por defecto un enrutador avanzado, un cliente HTTP integrado, herramientas de validación de formularios y un sistema de inyección de dependencias muy similar al utilizado en el *Backend* con Spring.

El modelo de componentes de Angular ha permitido dividir la compleja interfaz de SPIRITBLADE en piezas reutilizables y testables de forma aislada (por ejemplo, tarjetas de historial de partidas o visualizadores de perfil). La versión 17 destaca por la optimización del proceso de renderizado y la introducción de una sintaxis de control de flujo en las plantillas más intuitiva y eficiente.

HTML5 y CSS3

Para el marcado semántico y la estilización de la interfaz, se ha recurrido a los estándares web actuales: HTML5 y CSS3. Se ha aplicado un diseño *responsive* (adaptativo) apoyado en módulos de diseño como *Flexbox* y *CSS Grid*, garantizando que la visualización de las métricas de partidas y los elementos visuales (como los iconos

de los campeones y el historial de victorias/derrotas) se representen de forma óptima independientemente del dispositivo y la resolución de pantalla del usuario.

4.1.3. Almacenamiento y Persistencia de Datos

MySQL

Toda la información persistente generada en la aplicación, excluyendo la extraída en tiempo real de servicios externos, se almacena en una base de datos relacional utilizando MySQL [6]. Se trata de un Sistema de Gestión de Bases de Datos Relacionales (RDBMS, del inglés extitRelational Database Management System) maduro, de código abierto y ampliamente estandarizado.

MySQL es el responsable de almacenar de manera estructurada los registros de usuarios, contraseñas encriptadas, roles de autorización, listas de seguimiento (*favoritos*) y configuraciones de perfiles. Su perfecta integración con el ecosistema de Spring Data JPA y su fiabilidad transaccional justifican su inclusión en la arquitectura del proyecto.

4.1.4. Tecnologías de Seguridad Web

JSON Web Tokens (JWT)

En sistemas distribuidos modernos, mantener el estado de la sesión en el servidor resulta ineficiente e imposibilita la correcta escalabilidad horizontal (un aspecto que se abordará de forma exhaustiva en el segundo TFG). Para resolver esto, el módulo de autenticación de SPIRITBLADE emplea JWT [7].

Una vez que un usuario valida sus credenciales exitosamente a través de `LoginRestController`, el *Backend* genera un *token* codificado y firmado criptográficamente que es remitido al cliente. En las subsiguientes interacciones, el cliente de Angular adjunta dicho *token* en la cabecera HTTP de autorización. Esto permite al servidor validar la identidad del usuario y sus permisos de manera *stateless* (sin estado), sin necesidad de consultar repetidamente la base de datos o mantener variables en memoria.

4.1.5. APIs y Servicios de Terceros

Para proveer el núcleo de valor analítico y de contenido a SPIRITBLADE, ha sido indispensable la comunicación asíncrona con plataformas de terceros.

API Oficial de Riot Games

El suministro primario de información sobre *League of Legends* proviene directamente de la *Riot Games Developer API* [8]. Esta interfaz pública RESTful permite a los desarrolladores registrados acceder a grandes volúmenes de datos actualizados. En SPIRITBLADE, se consumen múltiples *endpoints* de esta API para obtener los perfiles de invocador, historiales de partidas en formato JSON (*Match-V5*), puntuaciones de visión, ratios de KDA y niveles de maestría de los campeones.

API de Google Gemini

Para implementar las funcionalidades de Inteligencia Artificial que generan resúmenes analíticos del rendimiento del jugador en sus últimas partidas, se ha integrado la API de los modelos fundacionales de Google Gemini [9]. El *backend* actúa como intermediario seguro: recopila el contexto numérico y estadístico del jugador, construye un *prompt* estructurado, invoca la API de Gemini y procesa la respuesta en lenguaje natural para presentarla en el *Frontend*.

4.2. Herramientas

El desarrollo de *software* moderno requiere de un ecosistema de herramientas de apoyo que faciliten la escritura de código, automaticen tareas repetitivas, aseguren la calidad del producto y permitan llevar un control exhaustivo de los cambios. A continuación, se detallan las principales herramientas utilizadas durante la construcción de SPIRITBLADE.

4.2.1. Control de Versiones y Alojamiento

Git y GitHub

Para el control de versiones del código fuente se ha empleado Git [10]. Esta herramienta distribuida es el estándar de la industria y permite registrar el historial de modificaciones, facilitando la experimentación segura mediante el uso de ramas (*branches*) y la recuperación de versiones estables en caso de introducir errores críticos en el código.

El repositorio remoto se ha alojado en GitHub. Más allá de servir como una copia de seguridad en la nube, GitHub ha proporcionado la infraestructura necesaria para la

revisión de código a través de *Pull Requests* (PR), la gestión de incidencias (*Issues*) y la ejecución de flujos de trabajo automatizados, siendo una pieza clave en la metodología de desarrollo del proyecto.

4.2.2. Gestión de Dependencias y Construcción

Dado que el proyecto se divide en dos módulos independientes (cliente y servidor), se han utilizado gestores de paquetes específicos para los ecosistemas de Java y Node.js.

Apache Maven

Para el *Backend*, se ha utilizado Maven [11] como herramienta de gestión de dependencias y comprensión del proyecto. Maven estandariza la estructura del directorio de Java (separando el código fuente de los recursos y las pruebas) y descarga automáticamente las librerías declaradas en el archivo `pom.xml` (como Spring Boot, librerías de JWT, conectores de MySQL y dependencias de *testing*) desde el repositorio central. Además, facilita la ejecución de las fases del ciclo de vida del *software*, como la compilación, la ejecución de pruebas unitarias y el empaquetado final en un archivo ejecutable `.jar`.

Node Package Manager (npm)

En el lado del *Frontend*, se ha empleado `npm` [12], el gestor de paquetes por defecto del entorno Node.js. A través del archivo `package.json`, `npm` administra todas las dependencias necesarias para el funcionamiento de Angular, así como librerías de terceros (por ejemplo, para la generación de gráficos o utilidades de diseño). También se ha utilizado para definir y ejecutar *scripts* de construcción y despliegue del servidor de desarrollo local.

4.2.3. Entornos de Desarrollo Integrados (IDEs)

La elección de un buen IDE impacta directamente en la productividad y la detección temprana de errores semánticos.

Evolución del Entorno de Desarrollo: de VS Code a Antigravity

En las etapas iniciales del proyecto, el desarrollo se centralizó exclusivamente en **Visual Studio Code (VS Code)**. La elección respondió a su naturaleza como estándar de facto en la industria, destacando por su ligereza y su vasto ecosistema de extensiones que facilitaron la maquetación de la SPA con *Angular* y la integración de herramientas de análisis estático como *Sonarqube For IDE*.

No obstante, tras su lanzamiento en noviembre de 2025, se optó por incorporar **Antigravity** como IDE de apoyo fundamental. Esta decisión estratégica permitió aprovechar sus avanzados modelos de inteligencia artificial integrados, los cuales optimizaron significativamente las tareas de refactorización de código en Java e investigación.

La adopción de esta nueva herramienta no solo mejoró la productividad, sino que refleja un pilar clave en el desarrollo de *software* moderno: la **adaptación continua a los cambios del sector**. En un ecosistema tecnológico que evoluciona a un ritmo sin precedentes, la capacidad de evaluar e integrar soluciones emergentes —como los entornos de desarrollo potenciados por IA— resulta indispensable para mantener la competitividad y la calidad técnica del producto final.

4.2.4. Herramientas de Pruebas y Control de Calidad

Swagger (OpenAPI)

Durante la construcción y evolución de la API REST, la herramienta principal empleada para la documentación interactiva y la validación de los *endpoints* del servidor ha sido Swagger, mediante la adopción de la especificación OpenAPI. Gracias a la configuración nativa implementada en el ecosistema de Spring Boot (a través de la clase `OpenApiConfig`), el sistema expone de manera automatizada una interfaz gráfica accesible directamente desde el navegador web.

Esta plataforma ha permitido explorar de forma ágil e intuitiva la estructura de todos los controladores expuestos, construir peticiones HTTP (GET, POST, PUT, DELETE) y depurar las respuestas en formato JSON. Además, la interfaz se configuró adecuadamente para soportar el esquema de seguridad del proyecto, permitiendo la inyección dinámica de los *tokens* JWT para probar las rutas protegidas. Su utilización ha sido fundamental para certificar el correcto funcionamiento de la lógica de negocio, la comunicación asíncrona con la API de Riot Games y las evaluaciones devueltas por el servicio de Inteligencia Artificial de Gemini. De forma complementaria y marginal, se empleó Postman únicamente para la ejecución de pruebas aisladas en las fases más

tempranas del desarrollo. Las Figuras 4.1 y 4.2 ilustran la interfaz final de Swagger UI documentando la API del proyecto.

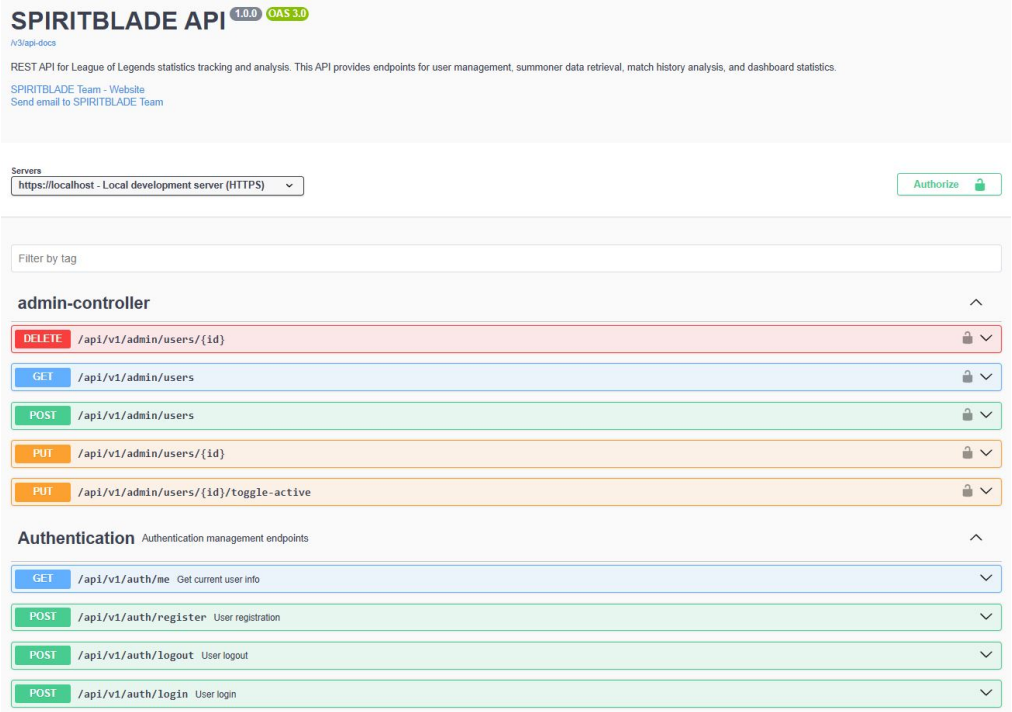


Figura 4.1: Interfaz interactiva de Swagger documentando la API REST de SPIRITBLADE. Se observa el catálogo general de controladores y la correcta distinción entre rutas públicas y protegidas mediante autorización JWT (indicadas con un candado).

SonarCloud

En el ámbito del QA, se ha integrado SonarCloud [13] como plataforma de inspección continua. Esta herramienta realiza un análisis estático del código fuente en cada *commit* o PR, identificando vulnerabilidades de seguridad, errores (*bugs*), redundancias y *code smells* (malas prácticas de programación). Garantizar que el código cumpla con las métricas de calidad impuestas por SonarCloud ha sido un requisito indispensable antes de fusionar cualquier cambio en la rama principal.

4.2.5. Automatización: Integración Continua (CI)

GitHub Actions

Para automatizar los procesos de validación, se ha hecho uso de GitHub Actions. Se han configurado diversos flujos de trabajo (*workflows*) en formato YAML (como `build.yml` y `build-push.yml`) que se disparan automáticamente ante determinados eventos en el repositorio.

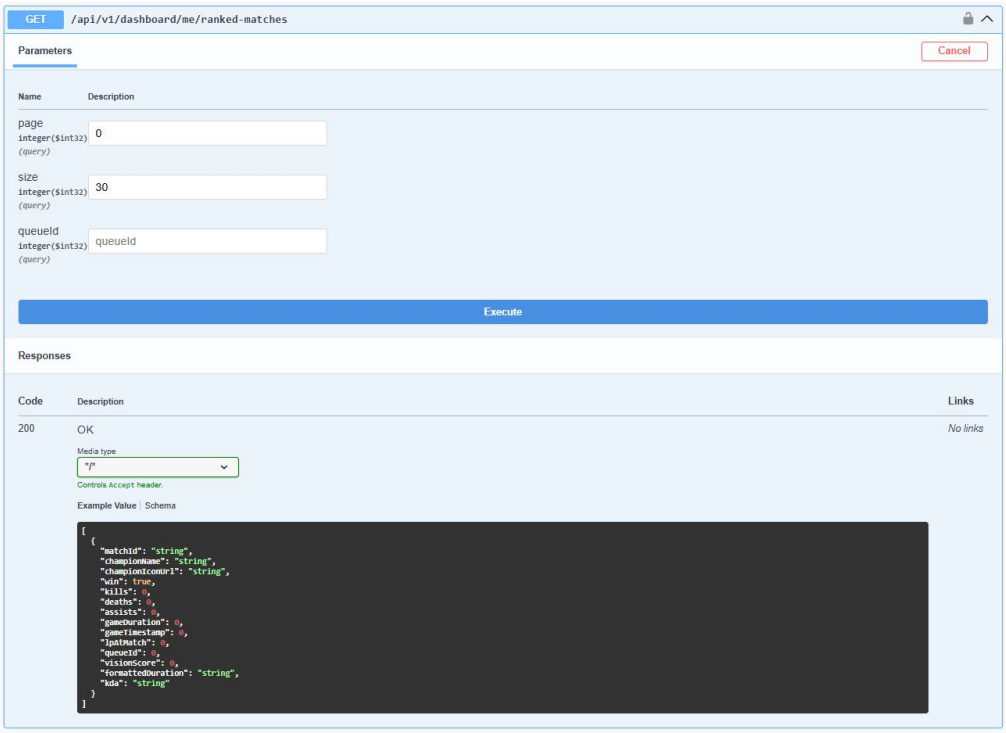


Figura 4.2: Detalle de un *endpoint* específico en Swagger UI, mostrando los parámetros requeridos, el esquema de respuesta JSON (DTO) y el botón de autorización.

En el contexto de este primer TFG, la responsabilidad de estos flujos abarca la **Integración Continua** (CI) y la **Entrega Continua** (CD). La fase de integración incluye la instalación de dependencias, la compilación del código (*Backend* y *Frontend*), la ejecución automatizada de la batería completa de pruebas y el análisis estático de código mediante SonarCloud. Durante las iteraciones más avanzadas del desarrollo, este *pipeline* adquirió una importancia crítica para certificar la estabilidad de los nuevos subsistemas arquitectónicos incorporados. Específicamente, el flujo CI garantiza la fiabilidad del nuevo sistema de caché mediante la ejecución de una cobertura ampliada en las pruebas unitarias de los servicios (tales como `DashboardServiceUnitTest` y `RiotServiceUnitTest`), validando rigurosamente las políticas de invalidación y actualización de datos en tiempo real. Adicionalmente, el pipeline asegura la integridad del almacenamiento distribuido de objetos multimedia mediante la ejecución continua de los escenarios de prueba dedicados a MinIO (`MinioStorageServiceUnitTest`), los cuales previenen cualquier posible regresión en la gestión de archivos. Si alguna comprobación falla, GitHub bloquea inmediatamente la integración del código. Por su parte, la entrega continua se encarga de la construcción y empaquetado del código en imágenes Docker listas para ser distribuidas. Como se ha reiterado, las acciones relativas al Despliegue Continuo (*Continuous Deployment*) en infraestructuras *cloud* quedan delegadas de forma exclusiva al segundo TFG.

4.2.6. Virtualización para Desarrollo Local

Docker y Docker Compose

Aunque el despliegue en entornos de producción mediante contenedores está fuera del alcance de este documento, se ha utilizado Docker [14] y Docker Compose con fines puramente orientados al entorno de desarrollo local.

Mediante la definición de un archivo `docker-compose.yml`, se ha podido levantar de forma orquestada y reproducible un contenedor efímero con la base de datos MySQL en la máquina del desarrollador. Esto aísla el entorno de datos del sistema operativo anfitrión, evita la necesidad de instalar el motor de base de datos localmente y garantiza que todos los desarrollos y pruebas se ejecuten sobre una configuración idéntica.

4.3. Arquitectura

El diseño arquitectónico de SPIRITBLADE se fundamenta en un patrón Cliente-Servidor fuertemente desacoplado, estandarizado mediante la exposición de una API RESTful. Esta separación arquitectónica no solo facilita el desarrollo en paralelo y la especialización de las tecnologías de *frontend* y *backend*, sino que es un requisito indispensable para las futuras fases de escalabilidad horizontal y despliegue en contenedores que se abordarán en el segundo TFG (ver Figura 4.3).

4.3.1. Arquitectura del Servidor (Backend)

El servidor, desarrollado en Spring Boot, actúa como el orquestador principal de la plataforma. Su arquitectura interna sigue el patrón tradicional de diseño por capas (*Layered Architecture*), lo que garantiza una clara separación de responsabilidades:

- **Capa de Presentación (Controladores):** Encargada de interceptar las peticiones HTTP entrantes. Clases como `SummonerController`, `UserController` o `DashboardController` gestionan los flujos de entrada, validan los parámetros y devuelven los códigos de estado apropiados junto con los DTOs en formato JSON.
- **Capa de Negocio (Servicios):** Contiene la lógica central de la aplicación, ejecutando los algoritmos de procesamiento de datos y orquestando las llamadas a las APIs externas. Para garantizar un diseño mantenible y cumplir rigurosamente con los principios SOLID (en especial, el Principio de Inversión de Dependencias),

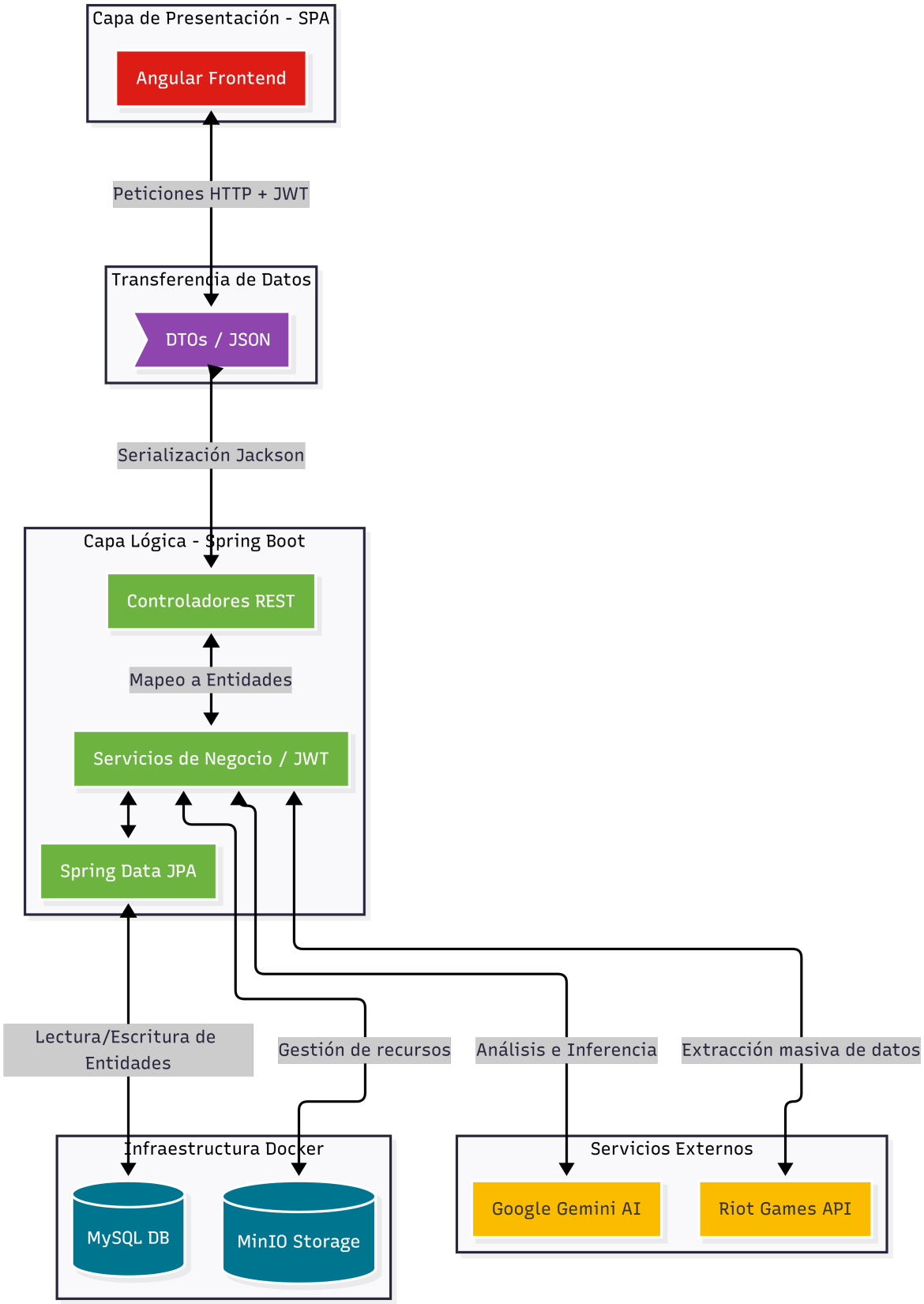


Figura 4.3: Diagrama de arquitectura y flujo de comunicación entre la SPA y el ecosistema de APIs externas.

el acceso a esta lógica se ha abstraído mediante el uso de interfaces. Esta decisión arquitectónica reduce drásticamente el acoplamiento entre los controladores y las implementaciones concretas de los servicios, delegando la resolución de estas dependencias al contenedor de Inversión de Control (IoC) de Spring Boot. Además, este bajo acoplamiento es lo que ha viabilizado la estrategia de *mocking* durante las pruebas unitarias comentada en apartados posteriores.

- **Capa de Acceso a Datos (Repositorios):** Abstrae la comunicación con la base de datos MySQL mediante interfaces de Spring Data JPA, aislando a la capa de negocio de los detalles del lenguaje SQL subyacente.

4.3.2. Patrones de Diseño: DTO y Mappers

Para garantizar un acoplamiento débil entre la capa de persistencia y la capa de presentación, la arquitectura del servidor hace uso extensivo del patrón estructural *Data Transfer Object* (DTO). La adopción de este patrón resulta crítica en la aplicación por dos motivos fundamentales:

- **Seguridad y encapsulamiento:** Evita la exposición directa de las entidades de la base de datos (como `UserModel` o `MatchEntity`) al cliente de Angular. Esto previene vulnerabilidades de seguridad severas, como la sobreasignación masiva de parámetros (*Mass Assignment*), y oculta los detalles de implementación de la persistencia relacional.
- **Eficiencia en la transferencia:** Permite empaquetar y transferir únicamente la información estrictamente necesaria para cada vista del *frontend*, optimizando el consumo de ancho de banda.

La responsabilidad de la conversión bidireccional entre las Entidades de dominio y los DTOs recae sobre clases utilitarias conocidas como *Mappers* (por ejemplo, `RiotMatchMapper` o `SummonerMapper`). Dado que SPIRITBLADE actúa como intermediario de las densas respuestas JSON provenientes de la API de Riot Games, estos *mappers* se encargan de filtrar el ruido, extraer exclusivamente las métricas analíticas relevantes (como asesinatos, muertes o visión) y traducirlas a las estructuras internas del sistema, manteniendo los controladores REST limpios y enfocados en la gestión del protocolo HTTP.

4.3.3. Arquitectura del Cliente (Frontend)

El cliente se ha construido como una SPA con Angular. La arquitectura en el navegador se basa en una jerarquía de componentes reactivos. El estado de la interfaz se actualiza dinámicamente gracias a los servicios inyectados de Angular, que consumen la API del *backend* de manera asíncrona y actualizan el Modelo de Objetos del Documento (DOM, del inglés extitDocument Object Model) sin necesidad de recargar la página, proporcionando la experiencia inmersiva e ininterrumpida que requieren las plataformas de análisis competitivo.

4.3.4. Consideraciones Arquitectónicas y Riesgos (Dependencia de Terceros)

Una de las características definitorias de la arquitectura de SPIRITBLADE es su condición de plataforma agregadora e intérprete de datos. El sistema depende intrínsecamente de dos grandes proveedores externos: Riot Games (para la extracción en crudo de las estadísticas) y Google Gemini (para el procesamiento de IA).

Si bien esta integración permite delegar responsabilidades de cálculo y almacenamiento masivo, supone la asunción de riesgos arquitectónicos significativos que han debido contemplarse durante el diseño:

- **Disponibilidad y Tolerancia a Fallos:** La plataforma está supeditada al estado ("*uptime*") de los servidores de Riot Games y de los servicios de IA. Una caída generalizada en la infraestructura de Internet puede interrumpir directamente el flujo de valor principal de la aplicación, como se ha evidenciado en incidentes a gran escala que han afectado simultáneamente a múltiples servicios, incluyendo League of Legends y plataformas de Inteligencia Artificial [15]. Para mitigar esto, se han implementado manejadores de excepciones (`GlobalExceptionHandler`) que capturan respuestas erróneas de los proveedores y ofrecen mensajes de error controlados al usuario, evitando que el cliente colapse.
- **Límites de Peticiones (*Rate Limiting*):** Las APIs externas imponen cuotas estrictas de peticiones por segundo y por minuto. Superar estos límites conlleva el bloqueo temporal de las claves de acceso.
- **Costes Operativos Inesperados:** Al integrar servicios de Inteligencia Artificial (Google Gemini), el volumen de *tokens* de entrada y salida genera costes que pueden escalar rápidamente frente a picos de tráfico no previstos, obligando a monitorizar activamente el uso de dichos servicios.

4.3.5. Modelo de dominio

El modelo de dominio de SPIRITBLADE se ha diseñado utilizando un enfoque centrado en la persistencia relacional a través de Spring Data JPA e Hibernate. Este modelo define las entidades principales que representan el núcleo del negocio y la forma en que se relacionan entre sí dentro de la base de datos MySQL.

A diferencia de otras plataformas, la base de datos en SPIRITBLADE no solo persiste información administrativa, sino que actúa como una capa de caché de primer nivel para reducir drásticamente el volumen de peticiones hacia las APIs externas, mejorando así los tiempos de respuesta. En las Figuras 4.4, 4.6 y 4.5 (ubicadas al final de esta sección) se ilustra el esquema de este modelo dividido por dominios, y a continuación, se describen las entidades fundamentales implementadas:

4.3.6. Entidad `UserModel`

Representa a los usuarios registrados en la plataforma. Más allá de almacenar las credenciales básicas (nombre de usuario, correo electrónico, contraseña encriptada y estado de actividad), esta entidad gestiona de manera compleja el contexto de personalización del jugador:

- **Roles de Seguridad:** Colección de roles asignados al usuario (como `USER` o `ADMIN`) para la autorización de acceso a rutas protegidas.
- **Vinculación de Cuenta:** Almacena el `puuid` y la región del invocador de *League of Legends* asociado al usuario, permitiendo ofrecer un *dashboard* de estadísticas personales.
- **Seguimiento de Invocadores:** Define una relación de muchos a muchos (`@ManyToMany`) con la entidad `Summoner`, posibilitando que un usuario mantenga una lista de jugadores favoritos para agilizar sus futuras búsquedas.
- **Gestión y Control de IA:** Controla la cuota de peticiones para evitar el abuso económico de la API de Google Gemini mediante el registro del momento exacto del último análisis solicitado (`lastAiAnalysisRequest`).

4.3.7. Consideraciones de Rendimiento: Desacoplamiento del Catálogo de Campeones

Al analizar la estructura relacional del modelo de dominio, resulta pertinente justificar una decisión de diseño arquitectónico fundamental en la capa de persistencia: la ausencia de una relación estricta de clave foránea (FK, del inglés extitForeign Key) y navegación orientada a objetos entre las entidades `MatchEntity` y `Champion`. En lugar de delegar el *mapping* al proveedor JPA mediante anotaciones tradicionales (como `@ManyToOne`), la entidad de la partida almacena de forma directa el identificador (`championId`) como un valor entero.

Esta estrategia de desnormalización responde a motivos críticos de eficiencia y escalabilidad:

- **Optimización de consultas (Lectura):** La tabla `champions` actúa conceptualmente como un diccionario o catálogo estático de datos (*Lookup Table*). Dado que la carga del historial de un jugador implica recuperar decenas o cientos de partidas simultáneamente, forzar al motor relacional a ejecutar operaciones de cruce de tablas (*JOIN*) por cada registro penalizaría drásticamente el tiempo de respuesta y el consumo de CPU de la base de datos.
- **Eficiencia en la ingesta de datos (Escritura):** Al procesar las masivas respuestas JSON provenientes de la API de Riot Games, el sistema recibe el `championId` numérico de forma directa. Almacenarlo en crudo evita la sobrecarga computacional de tener que realizar una consulta de validación (`SELECT`) para instanciar un objeto `Champion` en memoria antes de poder persistir la partida (`INSERT`).

Mediante este desacoplamiento, la resolución de los metadatos visuales (nombres de los personajes o rutas de las imágenes) se delega estratégicamente, resolviéndose en la capa de cliente (Angular), lo que preserva la integridad del rendimiento del servidor *backend*.

4.3.8. Entidad `Summoner`

Actúa como la representación persistente de un jugador del videojuego. Almacena identificadores críticos y únicos proveídos por la API de Riot Games (`riotId` y `puuid`) junto con métricas de rendimiento estáticas como el nivel actual, el icono de perfil y la clasificación en el modo competitivo (`tier`, `rank`, `lp`, `wins`, `losses`). Como se ha mencionado, su persistencia actúa de caché para evitar agotar las cuotas (*Rate Limits* o límites de tasa) impuestas por Riot Games.

4.3.9. Entidad `MatchEntity`

Constituye una de las entidades con mayor volumen transaccional, ya que almacena los detalles individuales de cada partida jugada por un invocador. Se relaciona mediante una cardinalidad de muchos a uno (`@ManyToOne`) con `Summoner`. Traduce a la base de datos los indicadores de rendimiento extraídos de respuestas JSON masivas, guardando únicamente el valor analítico:

- **Métricas de combate y economía:** Asesinatos, muertes, asistencias, daño total infligido, puntuación de visión y oro obtenido.
- **Metadatos del encuentro:** Duración de la partida, modo de juego, posición/rol desempeñado y el identificador del campeón seleccionado.

4.3.10. Entidad `RankHistory`

Cabe destacar que la concepción de esta entidad surge como respuesta arquitectónica a uno de los mayores desafíos técnicos abordados durante el desarrollo del proyecto. Durante la fase de integración con los servicios de terceros, se constató que la API oficial de Riot Games presenta una limitación significativa de diseño: no expone la fluctuación de Puntos de Liga (LP, del inglés extitLeague Points) —la unidad de puntuación competitiva que determina el progreso y ascenso de los jugadores dentro del sistema clasificatorio— asociada a cada partida individual ni conserva un registro histórico del estado clasificatorio de los invocadores. El proveedor externo se limita exclusivamente a facilitar el valor de los LP actuales del jugador en tiempo real.

Para subsanar esta estricta carencia y poder ofrecer una funcionalidad analítica avanzada de seguimiento de progresión, se implementó la entidad `RankHistory`. Su propósito fundamental es capturar y almacenar de manera persistente una "fotografía" (*snapshot*) del estado clasificatorio del jugador cada vez que se detecta y procesa un cambio en su historial.

Al vincularse mediante una relación opcional de uno a uno (`@OneToOne`) con la entidad `MatchEntity` que originó la alteración de la puntuación, el sistema subyacente de SPIRITBLADE es capaz de inferir, calcular y representar gráficamente las ganancias o pérdidas de LP a lo largo del tiempo. Esta decisión de diseño permite independizar la lógica analítica de la aplicación frente a las restricciones históricas impuestas por la API original.

4.3.11. Entidad `Champion`

Mantiene un registro unificado y estático del catálogo de personajes jugables del videojuego, incluyendo identificadores numéricos, nombres legibles y rutas hacia los recursos gráficos (`imageUrl`), simplificando la renderización de la interfaz en el *frontend*.

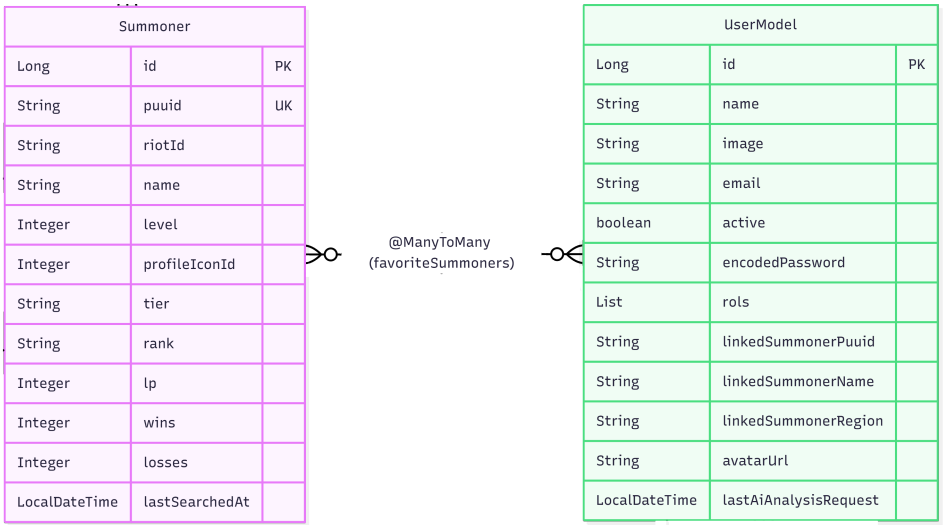


Figura 4.4: Diagrama de clases del dominio: Relación entre el Usuario (`UserModel`) y el Invocador (`Summoner`).

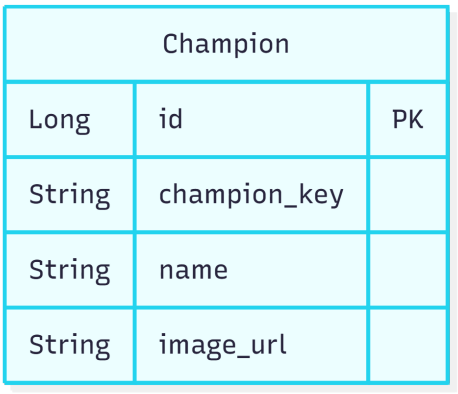


Figura 4.5: Diagrama de clases del dominio: Catálogo de Campeones (`Champion`) desacoplado del historial.

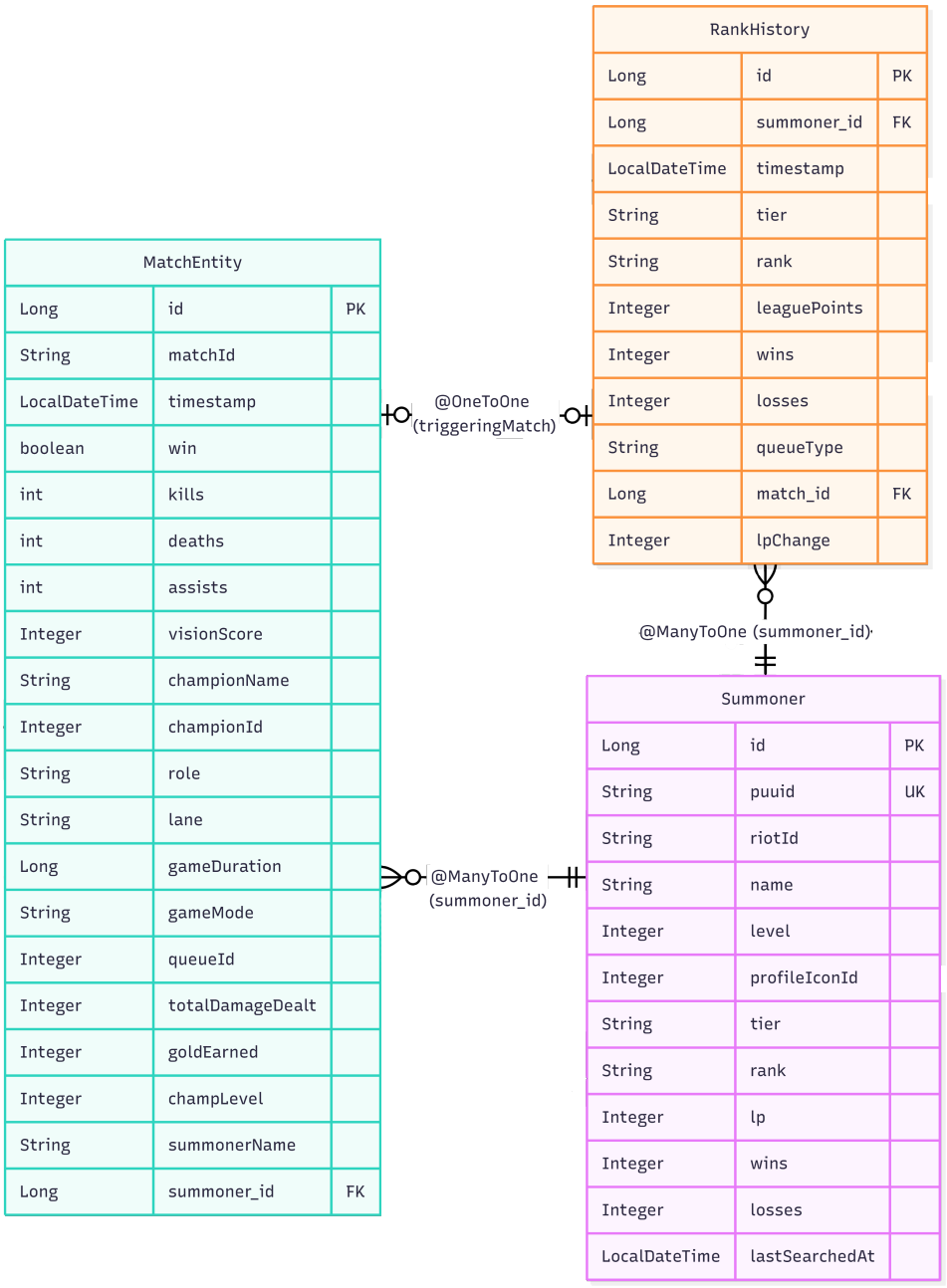


Figura 4.6: Diagrama de clases del dominio: Relación del Invocador con el historial de partidas (`MatchEntity`) y la progresión de rango (`RankHistory`).

4.4. Implementación

En esta sección se profundiza en los aspectos técnicos más relevantes y diferenciadores de la implementación de SPIRITBLADE, destacando las soluciones adoptadas para superar los retos específicos del proyecto y las decisiones arquitectónicas que han definido su evolución.

4.4.1. Integración con Riot Data Dragon

Para evitar el almacenamiento masivo de activos estáticos (iconos de campeones, objetos y hechizos), se ha implementado el servicio `DataDragonService`. Este componente actúa como un gestor de caché dinámico que construye las URLs oficiales de los activos de Riot Games en tiempo real, asegurando que la interfaz siempre muestre la versión más reciente del videojuego sin penalizar el almacenamiento del servidor.

4.4.2. Arquitectura de Seguridad y Gestión de Errores

Se ha implementado una capa transversal de seguridad basada en un filtro de interceptación (`JwtAuthenticationFilter`). Este valida la firma de cada token en cada petición antes de que llegue a la lógica de negocio.

Complementariamente, para garantizar la robustez del sistema, se ha desarrollado un **manejador global de excepciones** mediante la anotación `@ControllerAdvice`. Este mecanismo captura fallos imprevistos (como caídas de la API externa o errores de validación) y los transforma en respuestas JSON estandarizadas, mejorando la experiencia de usuario y facilitando la depuración en el *frontend*.

4.4.3. Algoritmo de Reconstrucción de LP y Sincronización Diferencial

Uno de los retos técnicos más significativos del proyecto surge de una limitación de la API de Riot Games: esta solo proporciona el estado actual de los LP, pero no su evolución histórica por partida. Para ofrecer visualizaciones de progreso, se ha desarrollado en el servicio `DashboardService` un algoritmo de reconstrucción retrospectiva.

El proceso se divide en dos estrategias clave:

1. **Sincronización Diferencial:** Mediante el método `checkIfCacheNeedsUpdate`, el

sistema compara el identificador de la última partida almacenada localmente con el obtenido en tiempo real. Solo si los identificadores difieren, se inicia el proceso de descarga, minimizando el consumo de la cuota de peticiones de la API (*rate limiting*, o limitación de tasa).

2. **Reconstrucción Inversa de LP:** Partiendo del LP actual del invocador, el método `calculateBackwardsLpChange` recorre cronológicamente hacia atrás el historial de partidas. Por cada victoria, se restan puntos del acumulado, y por cada derrota se suman, gestionando adicionalmente la lógica de promociones y descensos entre divisiones (saltos de 100 LP).

A continuación, se detalla en el Código 4.1 el fragmento central que implementa la recursividad e iteración de este retroceso temporal:

Código 4.1: Algoritmo de iteración y cálculo inverso de Puntos de Liga (LP).

```
1  private LpCalculationResult buildLpMap(List<MatchHistoryDTO> sortedMatches,
2          Map<String, MatchEntity> existingMatches, Summoner summoner,
3          String currentTier, String currentDivision, int currentLP) {
4      Map<String, Integer> lpByMatchId = new HashMap<>();
5      List<MatchEntity> newMatches = new ArrayList<>();
6      int lpTracker = currentLP;
7      String divisionTracker = currentDivision;
8
9      // Iteracion cronologica inversa sobre el historial
10     for (int i = sortedMatches.size() - 1; i >= 0; i--) {
11         MatchHistoryDTO matchDTO = sortedMatches.get(i);
12         MatchEntity existing = existingMatches.get(matchDTO.getMatchId());
13         Optional<Integer> existingLp = existing != null ?
               rankHistoryService.getLpForMatch(existing.getId()) : Optional.empty();
14
15         if (existingLp.isPresent()) {
16             lpTracker = existingLp.get();
17             lpByMatchId.put(matchDTO.getMatchId(), existingLp.get());
18             continue;
19         }
20
21         int lpAtMatchStart = lpTracker;
22         boolean won = matchDTO.getWin() != null && matchDTO.getWin();
23
24         // Calculo de retroceso de LP por partida
25         LpAndDivision calcResult = calculateBackwardsLpChange(lpTracker, won,
               currentTier, divisionTracker);
26         lpTracker = calcResult.lp();
27         divisionTracker = calcResult.division();
28
29         lpByMatchId.put(matchDTO.getMatchId(), lpAtMatchStart);
30         newMatches.add(MatchMapper.toEntity(existing, matchDTO, summoner));
31     }
32     return new LpCalculationResult(lpByMatchId, newMatches);
33 }
```

Este enfoque permite al sistema SPIRITBLADE generar gráficos de tendencia compe-

titiva de alta fidelidad sin que la fuente de datos original proporcione dicha información de forma nativa, aportando una ventaja competitiva y técnica sobre otras herramientas del mercado.

4.4.4. Algoritmo de Procesamiento y Análisis mediante IA

El componente `AiAnalysisService` es responsable de invocar al modelo Gemini 2.5 Flash para transformar las métricas crudas del jugador en consejos tácticos. El verdadero núcleo de esta integración reside en la **Ingeniería de Prompts** (*Prompt Engineering*).

Antes de realizar la petición, el sistema realiza una serialización exhaustiva del historial de partidas, transformando datos puramente numéricos (como el KDA, oro, daño, duración y rango de cada partida individual) a un formato de texto estructurado y fácilmente digerible por el modelo. Posteriormente, se inyecta toda esta información dentro de un *System Prompt* directivo.

Esta estrategia de refinado extremo y enriquecimiento del contexto previo a la consulta principal es una técnica avanzada ampliamente documentada y empleada por grandes modelos del mercado, como Claude de Anthropic [16], para lograr que la Inteligencia Artificial asuma una personalidad (o rol) específica y devuelva respuestas mucho más precisas, fundamentadas empíricamente [17] y estrictamente acordes al dominio del problema. En el Código 4.2 se ilustra la construcción de este contexto.

Código 4.2: Construcción del contexto y el System Prompt para la API de Gemini.

```
1  private String buildStatsPrompt(Summoner summoner, List<MatchEntity> matches) {
2      int totalMatches = matches.size();
3      long wins = matches.stream().filter(MatchEntity::isWin).count();
4      long losses = totalMatches - wins;
5      double winRate = (wins * 100.0) / totalMatches;
6
7      // Serializacion del contexto de hasta 50 partidas a lenguaje natural
8      final int MAX_MATCH_DETAILS = 50;
9      int detailLimit = Math.min(matches.size(), MAX_MATCH_DETAILS);
10     StringBuilder matchDetails = new StringBuilder();
11     for (int i = 0; i < detailLimit; i++) {
12         matchDetails.append(formatMatchDetails(matches.get(i), i + 1));
13     }
14
15     // Inyeccion de la personalidad y directrices de formato (System Prompt)
16     return String.format(
17             """
18             You are SPIRITBLADE, Shen's mystical blade and expert League of Legends
                   analyst.
19             Analyze the performance of player "%s" based on the following detailed
                   data from their last %d matches:
20
21             OVERALL SUMMARY:
22             - Total Matches: %d
```

```
23              - Wins: %d (%.1f%%)
24              - Losses: %d (%.1f%%)
25
26              DETAILED MATCH-BY-MATCH HISTORY:
27              %s
28
29              Based on this complete match history data, provide a comprehensive
                    analysis in English that includes:
30              1. **Performance Overview**: Overall assessment of the player's current
                    form and recent trends
31              2. **Key Strengths**: What the player excels at (be specific with data
                    from individual matches)
32              3. **Critical Weaknesses**: Areas that need immediate attention (cite
                    specific match examples)
33              4. **Strategic Recommendations**: Concrete tactical advice to improve
                    gameplay
34              ...
35              """,
36              summoner.getName(), totalMatches, totalMatches, wins, winRate, losses,
37              (losses * 100.0) / totalMatches, matchDetails.toString());
38  }
```

4.5. Control de calidad

Para asegurar la robustez del código y minimizar la introducción de regresiones durante los ciclos iterativos, se ha implementado una estrategia de aseguramiento de la calidad (QA) apoyada en baterías de pruebas automatizadas. Dado que la integración continua (CI) exige que todo el código fusionado sea funcional, las pruebas se han convertido en la red de seguridad del proyecto.

4.5.1. Estrategia de Aseguramiento de la Calidad: La Pirámide de Pruebas

Para garantizar la robustez, mantenibilidad y fiabilidad de SPIRITBLADE, el proceso de desarrollo se ha regido por una estrategia de calidad basada en el concepto de la **Pirámide de Pruebas** [18]. Esta estructura jerárquica permite optimizar los recursos de computación y el tiempo de los desarrolladores, distribuyendo el esfuerzo de validación en tres capas diferenciadas según su nivel de aislamiento, velocidad y coste de ejecución.

1. **Capa de Pruebas Unitarias (Base):** Constituye el volumen más extenso de la suite de pruebas. En el *backend*, se han implementado mediante **JUnit 5** y **Mockito**, aislando la lógica de los servicios mediante el uso de *mocks*. Su objetivo es validar componentes individuales (como *mappers* y lógica de negocio) en

milisegundos. En el *frontend*, se han empleado **Jasmine** y **Karma** para probar componentes y tuberías (*pipes*) de Angular. En total, se han implementado 395 pruebas unitarias en el *backend* y 313 en el *frontend*.

2. **Capa de Pruebas de Integración (Cuerpo):** Con un volumen intermedio, se han desarrollado 100 pruebas que validan la interacción entre los diferentes módulos del sistema y las dependencias externas (como la base de datos o la caché). Se ha hecho uso de `@SpringBootTest` y `MockMvc` para verificar que los *endpoints* REST responden correctamente al ciclo completo de petición-respuesta, incluyendo la serialización de DTOs y la persistencia JPA.

3. **Capa de Pruebas E2E (Cima):** Representa el menor porcentaje (con un total de 15 pruebas agrupadas en 4 flujos completos automatizados) de la pirámide debido a su fragilidad y elevado coste temporal. Utilizando **Selenium WebDriver**, se han automatizado flujos críticos de usuario (como el proceso de registro, *login* y búsqueda de invocador), simulando el comportamiento real de un navegador para certificar que el sistema completo funciona de manera cohesionada.

Esta distribución garantiza que la mayoría de los errores se detecten en las capas inferiores (más rápidas y fáciles de depurar), cumpliendo con el **Quality Gate** impuesto en **SonarCloud**, que exige una cobertura de código (*code coverage*) superior al 80 % antes de permitir cualquier integración en la rama principal.

4.5.2. Pruebas Unitarias y el Desafío de la Simulación (*Mocking*)

Las pruebas unitarias se han centrado en validar aisladamente los componentes críticos de la lógica de negocio, haciendo uso de librerías como JUnit 5 y Mockito en el entorno de Java.

Como apunte metodológico y anecdótico derivado del diseño del sistema, resulta destacable el esfuerzo requerido para aislar esta lógica. Debido a que la plataforma se basa casi en su totalidad en consumir, transformar y presentar información generada y provista por plataformas de terceros, el desarrollo de las pruebas unitarias exigió una estrategia de simulación (*mocking*) masiva y exhaustiva.

En la práctica, casi todo el contexto que rodea a los servicios debió ser "burlado" o simulado. Evaluar el cálculo del rendimiento de un jugador sin realizar peticiones HTTP reales a la API de Riot o a los servidores de Gemini implicó crear complejas estructuras de datos JSON falsas (*mocks*) que imitasen los comportamientos de éxito,

fracaso y latencia de estos agentes externos. Esta necesidad intensiva de simulaciones puso de manifiesto de forma muy gráfica la tremenda dependencia que tiene el núcleo de la aplicación respecto a sus proveedores externos.

4.5.3. Análisis estático de código

Para garantizar la mantenibilidad y seguridad, se ha integrado SonarCloud como plataforma de inspección continua. Esta herramienta realiza un análisis estático del código fuente en cada *commit* o *Pull Request*, identificando vulnerabilidades de seguridad, errores (*bugs*), redundancias y *code smells* (malas prácticas de programación). Garantizar que el código cumpla con las métricas de calidad impuestas ha sido un requisito indispensable antes de fusionar cualquier cambio (ver Figura 4.7).

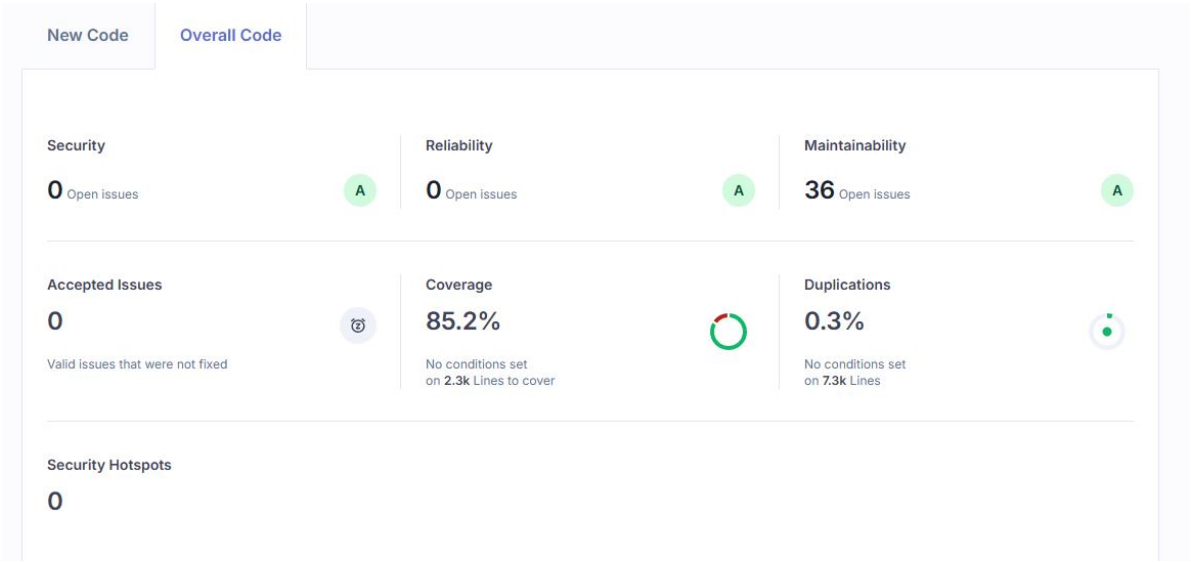


Figura 4.7: Panel de control de SonarCloud mostrando el estado satisfactorio del *Quality Gate* y métricas del tamaño del código.

4.6. Empaquetado y distribución

Dado el alcance de este primer TFG, el empaquetado y distribución de las versiones (*releases*) se ha realizado mediante la publicación de artefactos y contenedores Docker orientados a la evaluación local. El despliegue continuo automatizado en infraestructuras *cloud* conforma los trabajos futuros para el segundo TFG.

La aplicación se empaqueta en imágenes Docker alojadas en DockerHub, permitiendo un levantamiento íntegro del sistema mediante una única instrucción de orquestación

(`docker-compose up -d`). Esto facilita la labor de evaluación y garantiza la portabilidad del entorno.

El archivo `docker-compose.yml` define una infraestructura local autosuficiente compuesta por tres servicios principales que se comunican a través de una red virtual interna:

- **`spiritblade-app`:** Contenedor principal que aloja tanto la API REST (Spring Boot) como los archivos estáticos compilados de la aplicación cliente (Angular). Expone el puerto 443 (HTTPS) para el acceso desde el navegador, empleando certificados autofirmados para entornos de desarrollo. Este servicio depende directamente de la base de datos y espera a que esta se encuentre operativa (*healthcheck*) antes de iniciar su contexto de Spring.
- **`spiritblade-mysql`:** Instancia de la base de datos MySQL 8.0. Su almacenamiento se ha configurado mediante volúmenes de Docker (*Docker Volumes*) para asegurar la persistencia de los datos (usuarios, historiales y caché de la API) frente a la recreación o reinicio de los contenedores.
- **`spiritblade-minio`:** Servidor de almacenamiento de objetos compatible con el estándar S3, destinado a la gestión de imágenes de perfil subidas por los usuarios. Al igual que la base de datos, dispone de un volumen persistente anclado al sistema de archivos del servidor anfitrión.

El proceso de construcción (*build*) de la imagen principal `spiritblade-app` hace uso del patrón *Multi-stage Build* (Construcción Multi-etapa) de Docker. En una primera etapa, se utiliza un contenedor ligero de Node.js para instalar dependencias (`npm install`) y compilar la aplicación de Angular (`ng build`). En una segunda etapa, se utiliza una imagen de Maven y JDK 21 para compilar el *backend* de Spring Boot, copiando los archivos estáticos previamente generados por Angular en la carpeta de recursos estáticos del servidor web (`/src/main/resources/static/`). De este modo, el archivo ejecutable `.jar` final contiene tanto la lógica de servidor como la interfaz gráfica, optimizando el tamaño y la seguridad de la imagen de producción resultante.

4.7. Proceso de desarrollo

La materialización de la arquitectura y el modelo de dominio en código ejecutable se llevó a cabo aplicando el ciclo de vida iterativo detallado en el Capítulo 3. Para garantizar un desarrollo ordenado y auditable, la implementación se dividió en fases claramente definidas:

4.7.1. Fase 1: Análisis de Requisitos y Diseño de Interfaces

En esta fase inicial, el esfuerzo se centró en la conceptualización del producto. Se definieron los casos de uso principales y se diseñaron los prototipos estáticos de baja fidelidad (*wireframes*) utilizando HTML y CSS. Esto permitió estructurar vistas críticas como el *dashboard* interactivo, el perfil del invocador y el panel de administración. El objetivo primordial de esta fase fue fijar la identidad visual inmersiva de SPIRITBLADE y acordar el flujo de navegación antes de escribir cualquier lógica de programación.

4.7.2. Fase 2: Infraestructura Base, Repositorio y Pruebas

Se estableció el esqueleto técnico y metodológico de la aplicación:

- Inicialización del repositorio Git e implementación del modelo de ramificación *GitHub Flow*.
- Configuración inicial de los gestores de dependencias (Maven para Java y npm para Angular).
- Integración temprana de la herramienta de análisis de código estático (SonarCloud) y configuración de las *pipelines* de Integración Continua (GitHub Actions) para la validación automática de confirmaciones de código.

4.7.3. Fase 3: Funcionalidad básica y Docker

Esta primera iteración de desarrollo tuvo como objetivo principal establecer un Producto Mínimo Viable (MVP, del inglés *Minimum Viable Product*) completamente funcional desde el servidor hasta la interfaz del cliente. Los principales hitos de esta fase incluyeron la creación de los cimientos del *Backend* mediante `RiotService` para extraer perfiles de jugador y partidas básicas desde la API oficial, junto con la creación de los componentes base en Angular para visualizarlos de forma dinámica. De manera crucial, y a diferencia de enfoques tradicionales, durante esta misma etapa temprana se configuró la contenerización del entorno de base de datos local y servicios auxiliares mediante `docker-compose.yml`, garantizando así un entorno de ejecución estandarizado y reproducible para el resto del ciclo de vida del proyecto.

4.7.4. Fase 4: Funcionalidad intermedia y avanzada

Sobre la base tecnológica cimentada en la iteración previa, el esfuerzo se dirigió a implementar la lógica de negocio avanzada y cumplir con los requisitos de autorización. Se desarrolló el sistema completo de seguridad apoyado en JWT para gestionar sesiones de usuarios registrados de manera *stateless*. En el plano analítico, se construyó el núcleo diferenciador de la plataforma: la integración del motor de Inteligencia Artificial (`AiAnalysisService`) con la API de Google Gemini, traduciendo complejas estructuras de datos masivos en análisis tácticos naturalizados. Simultáneamente, el *Frontend* se enriqueció con funcionalidades interactivas profundas, como el panel de administración, la gestión de listas de invocadores favoritos y la representación gráfica avanzada de las fluctuaciones de rango (LP).

4.7.5. Fase 5: Escritura de la memoria

Esta fase se dedicó de manera exclusiva a la redacción, estructuración y revisión del presente documento académico. El proceso implicó una profunda labor de documentación retrospectiva para justificar formalmente cada una de las decisiones arquitectónicas, tecnológicas y de diseño tomadas durante la construcción del *software*. Se empleó LaTeX sobre la plataforma Overleaf para garantizar un formato riguroso y profesional, incorporando diagramas arquitectónicos, métricas de SonarCloud, fragmentos de código relevantes y capturas de la interfaz final que respaldan empíricamente el cumplimiento de los objetivos establecidos. Asimismo, se integró la retroalimentación continua del tutor para refinar la claridad expositiva y la precisión técnica del texto.

4.7.6. Fase 6: Preparación de la presentación

La fase de cierre del proyecto consistió en la elaboración integral de los materiales de soporte para la defensa pública del TFG ante el tribunal evaluador. Durante este periodo, se diseñó una presentación audiovisual orientada a sintetizar meses de ingeniería y desarrollo en un discurso claro, destacando los aportes de mayor valor (como el enriquecimiento de datos con IA) y la solidez arquitectónica de la plataforma implementada. Adicionalmente, se preparó y editó el vídeo demostrativo (*demo*) que ilustra los flujos de usuario, ensayándose además la ejecución interactiva en directo para validar empíricamente la madurez, fluidez y tolerencia a fallos del sistema ante las posibles cuestiones técnicas del comité.

Además de las fases iterativas mencionadas, el desarrollo se ha apoyado en prácticas

estandarizadas para la gestión y control del código fuente:

4.7.7. Gestión de tareas

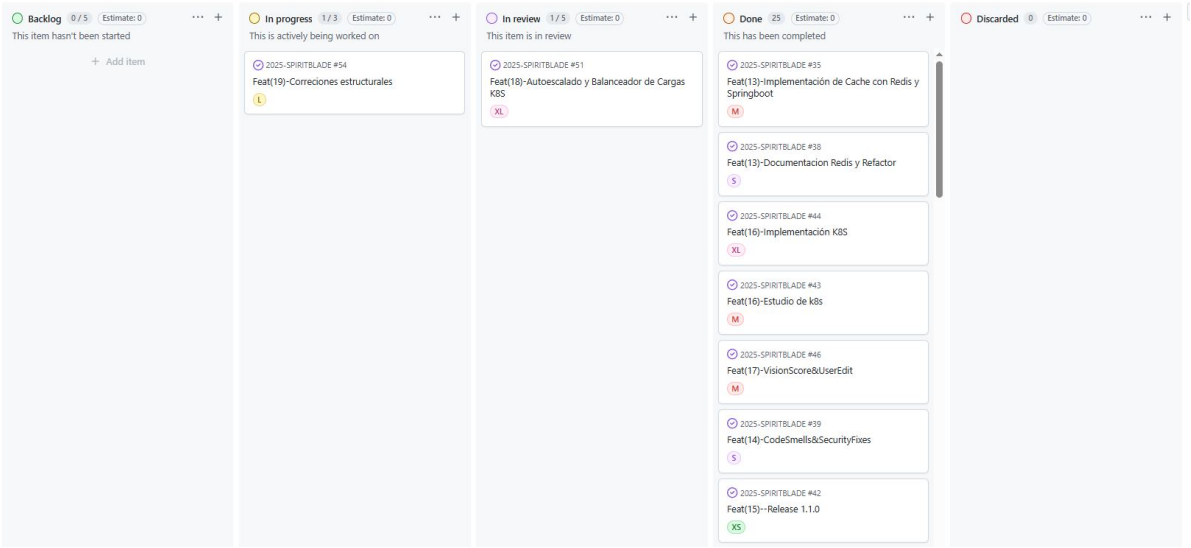


Figura 4.8: Vista detallada del *Backlog* del proyecto en GitHub Projects.

Para la organización diaria, priorización de requisitos y seguimiento del progreso del desarrollo, se adaptaron principios de metodologías ágiles, empleando concretamente el sistema **Kanban** [19].

A través de la herramienta *GitHub Projects*, integrada directamente con el repositorio del código fuente, se construyó un tablero visual. Este tablero permitió representar el flujo de valor del proyecto mediante tarjetas (tareas o *issues*) que transitaban por diferentes estados (columnas):

- **Backlog:** Pila de producto donde se listaban todas las tareas, funcionalidades futuras, mejoras e ideas pendientes de abordar, priorizadas de mayor a menor importancia. Como se observa en la Figura 4.8, las tareas cuentan con una estimación de esfuerzo basada en el tallaje de camisetas (*T-shirt sizing*). Estas estimaciones se determinaron mediante sesiones de *Planning Poker* realizadas en colaboración con inteligencia artificial, lo que permitió ajustar la complejidad de cada funcionalidad de manera objetiva.
- **To Do (Por hacer):** Tareas seleccionadas del *Backlog* para ser desarrolladas de manera inminente en la iteración actual.
- **In Progress (En progreso):** Tareas que se encontraban en fase de desarrollo activo. Se aplicó la regla fundamental de Kanban de limitar el trabajo en curso

(*Work In Progress* o WIP), fijando un límite máximo de 3 tareas concurrentes para evitar la dispersión de esfuerzos y garantizar que las tareas se cerraran antes de abrir nuevas.

- **In Review / Testing (En revisión):** Funcionalidades terminadas pero pendientes de superar los análisis estáticos de código y las baterías de pruebas automatizadas.
- **Done (Terminado):** Tareas completamente finalizadas, probadas e integradas en la rama principal del proyecto.

4.7.8. Git y Estrategia de Ramas

Para coordinar el desarrollo de forma metódica y mantener un historial de control de versiones rastreable, se implementó el modelo de ramificación **GitHub Flow**.

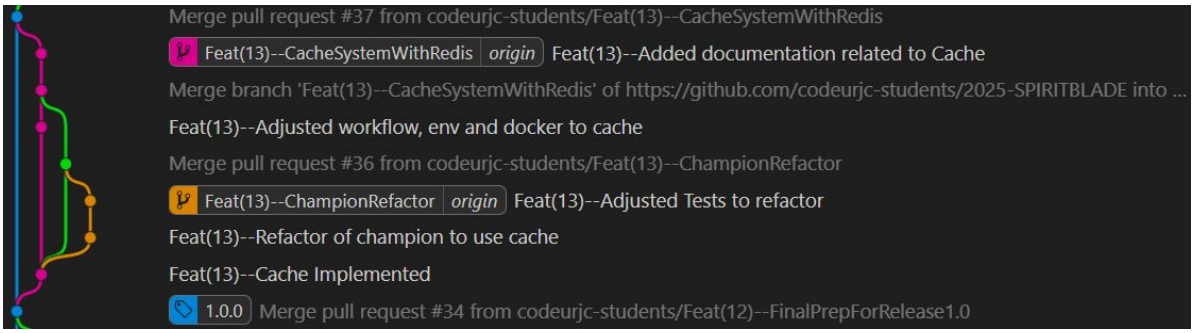


Figura 4.9: Representación visual del árbol de ramas siguiendo la metodología de features.

Como se muestra en la Figura 4.9, la estrategia adoptada consistió en utilizar la rama `main` como fuente única de verdad, y derivar ramas de funcionalidad (`feature branches`) por cada tarea registrada en el tablero Kanban.

4.7.9. Integración y entrega continua

Una vez finalizado el trabajo en una rama, se abría una petición de integración (*Pull Request*). La creación de este PR disparaba automáticamente los flujos de trabajo de GitHub Actions. En este entorno aislado, se ejecutaban las siguientes tareas automatizadas: instalación de dependencias, compilación del *Backend* y *Frontend*, ejecución de todos los *tests*, y lanzamiento del análisis estático mediante SonarCloud.

Para suplir la falta de revisión por pares (*peer review*), se configuró asistentes de revisión automatizados con IA para sugerir optimizaciones antes de la fusión, tal y

como se observa en las Figuras 4.10 y 4.11.

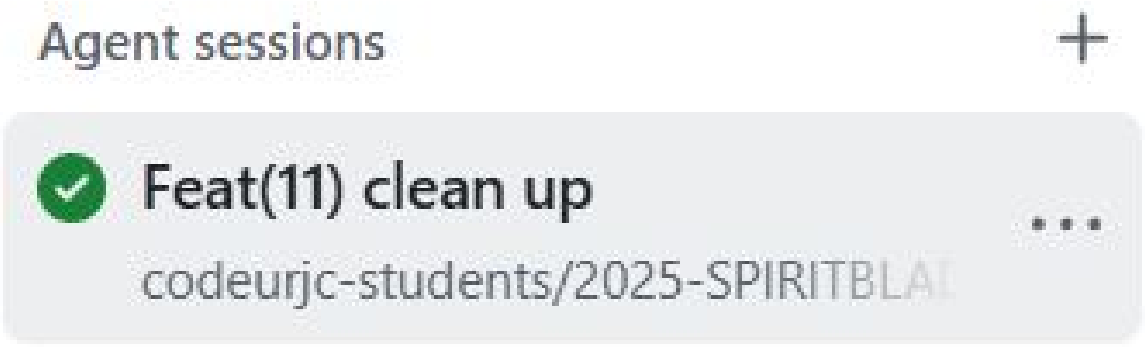


Figura 4.10: Ejemplo de revisión de código automatizada en los PR.

4.7.10. Versionado

Para la gestión de las entregas y la identificación del estado de madurez del *software*, se ha adoptado un enfoque pseudo-semántico fuertemente inspirado en el estándar de Versionado Semántico (*Semantic Versioning* o SemVer). Dado que la naturaleza de una aplicación web de cara al usuario final carece del concepto tradicional de cambios de rotura estricta (*breaking changes*) propio de las APIs públicas o librerías de terceros, este modelo se ha adaptado a la realidad del producto.

Bajo esta convención, cada incremento del sistema se etiqueta con un número de versión estructurado en el formato `MAYOR.MENOR.PARCHE` (N.N.N), donde cada componente posee un significado específico dentro de este ciclo de vida:

- **Versión MAYOR (*Major*):** Se incrementa al introducir cambios arquitectónicos drásticos, refactorizaciones de gran calado o hitos de entrega significativos (como el paso de *Beta* a producción).
- **Versión MENOR (*Minor*):** Se actualiza al añadir nuevas funcionalidades principales al producto.
- **Versión PARCHE (*Patch*):** Se eleva al implementar correcciones de errores (*bugfixes*), ajustes visuales y parches de seguridad.

No obstante, tras un análisis retrospectivo del repositorio, se identifica como un área de mejora la gestión de la granularidad en las versiones de tipo `MENOR`. En diversas etapas del ciclo de desarrollo, se detectó una tendencia a infrautilizar este nivel de versionado.

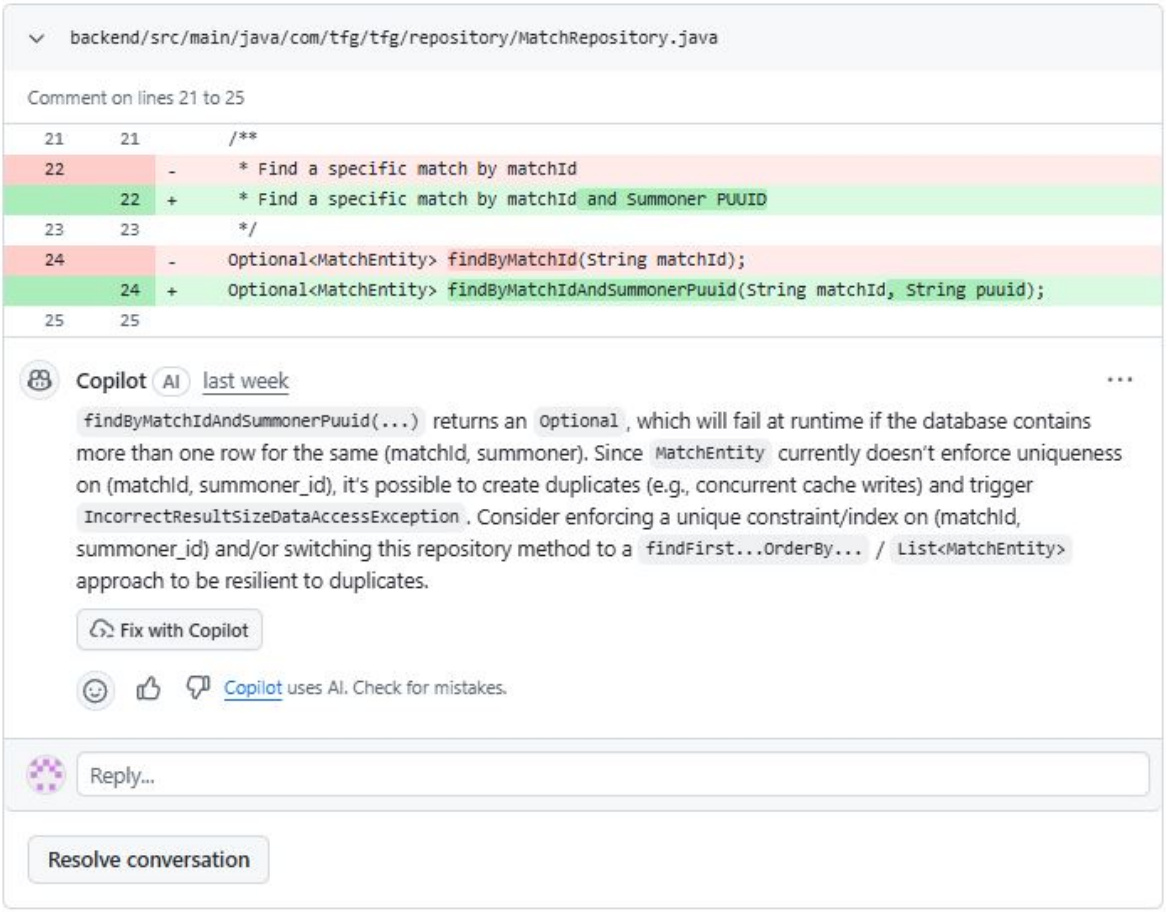


Figura 4.11: Ejemplo de sugerencias de mejora en un PR.

Esta desviación respecto al estándar ideal ha servido como un proceso de aprendizaje crítico sobre la importancia de la trazabilidad y la liberación frecuente de incrementos funcionales. Dicha lección se considera fundamental para la madurez profesional del desarrollador.

Dado el alcance de este primer TFG, el empaquetado y distribución de las versiones (*releases*) se ha realizado mediante la publicación de artefactos y contenedores Docker orientados a la evaluación local. El despliegue continuo automatizado en infraestructuras *cloud* conforma los trabajos futuros para el segundo TFG.

5

Conclusiones y trabajos futuros

En este capítulo final se presenta una reflexión global sobre el trabajo desarrollado a lo largo de este primer TFG. Se evalúa el grado de cumplimiento de los objetivos planteados inicialmente, se analizan los retos técnicos superados y se detallan las líneas de ampliación futuras, haciendo especial hincapié en la transición hacia el segundo proyecto exigido por la doble titulación. Finalmente, se exponen las conclusiones personales del autor respecto a la realización de esta memoria.

5.1. Conclusiones

En esta sección se sintetizan los resultados obtenidos, se reflexiona sobre el proceso de desarrollo y se extraen las lecciones aprendidas a lo largo de la implementación de SPIRITBLADE.

5.1.1. Consecución de objetivos funcionales y técnicos

Al inicio de este proyecto, se estableció como meta principal el diseño y desarrollo de SPIRITBLADE, una aplicación web *full-stack* orientada al análisis avanzado de datos de *League of Legends*. Tras concluir las fases de implementación y validación, se puede

afirmar que **el objetivo general se ha cumplido de forma satisfactoria**.

A nivel de objetivos específicos, se ha logrado implementar la totalidad de las funcionalidades propuestas para los distintos roles del sistema:

- **Módulo de acceso público:** Se ha construido con éxito un motor de búsqueda que interactúa en tiempo real con la API de Riot Games, permitiendo visualizar los perfiles de invocador, historiales de partidas y métricas de maestría de forma intuitiva.
- **Módulo privado y autenticación:** Se ha implementado un sistema robusto de cuentas de usuario protegido mediante JWT. Los usuarios pueden registrarse, iniciar sesión, vincular su cuenta de juego y gestionar sus listas de seguimiento (*favoritos*) de forma segura y persistente en la base de datos MySQL.
- **Innovación mediante inteligencia artificial:** Se ha cumplido el objetivo diferenciador del proyecto al integrar exitosamente la API de Google Gemini. El sistema es capaz de recopilar el contexto estadístico de las partidas de un jugador y generar informes analíticos automatizados, aportando un alto valor añadido frente a otras herramientas del mercado.
- **Panel de administración:** Se ha dotado a la plataforma de un panel de control funcional para la gestión y moderación de cuentas de usuario, asegurando la mantenibilidad de la comunidad de SPIRITBLADE.

Desde la perspectiva de la **ingeniería del *software***, se han superado los estándares de calidad exigidos. El código se ha estructurado bajo un patrón de arquitectura multicapa claro en Spring Boot y un diseño basado en componentes en Angular. Asimismo, se ha cumplido con el requisito de asegurar la calidad mediante la implementación de baterías de pruebas automatizadas (unitarias, de integración y E2E) y el análisis estático en la nube con SonarCloud, garantizando una base de código libre de vulnerabilidades críticas y con una baja deuda técnica.

5.1.2. Reflexión y lecciones aprendidas

El desarrollo de una aplicación web completa y su integración con múltiples servicios externos ha supuesto un desafío técnico de considerable envergadura. A lo largo de las iteraciones, se han extraído valiosas lecciones aplicables al ámbito del desarrollo de *software* profesional.

Uno de los mayores retos superados fue la **gestión de límites de la API de Riot Games** (*rate limiting*). Inicialmente, la gran cantidad de peticiones necesarias para construir un historial de partidas detallado provocaba bloqueos por parte del servidor externo. La resolución de este problema mediante la implementación de una **estrategia de caché persistente en la base de datos MySQL** no solo optimizó los tiempos de respuesta, sino que demostró la importancia de diseñar arquitecturas resilientes ante restricciones de red y de minimizar las llamadas redundantes a servicios de terceros.

Por otro lado, la **integración de modelos de Inteligencia Artificial generativa** demostró ser un área que requiere un cuidado especial en la fase de ingeniería de *prompts* (*prompt engineering*). Asegurar que Gemini devolviera respuestas estructuradas, consistentes y útiles a partir de datos puramente numéricos requirió múltiples refinamientos en la lógica del *backend*.

En conjunto, este proyecto ha reafirmado la eficacia de las metodologías iterativas e incrementales. La fragmentación del desarrollo en pequeñas entregas de valor, apoyadas por tableros Kanban y flujos de revisión estricta (*GitHub Flow*), ha sido clave para no perder el enfoque y mantener el progreso constante a pesar de las dificultades técnicas.

5.1.3. Conclusiones personales

A nivel personal y académico, la realización de este TFG ha representado la culminación de los conocimientos adquiridos durante los años de formación en la Universidad Rey Juan Carlos. Enfrentarse a la concepción, diseño y construcción desde cero de una plataforma *full-stack* supone un salto cualitativo desde los ejercicios académicos aislados hacia la realidad de la ingeniería de *software* moderna.

Este proyecto me ha permitido consolidar mis competencias en el ecosistema de Java y el desarrollo web con Angular, así como explorar tecnologías de vanguardia como la Inteligencia Artificial de Google Gemini. Más allá del código, me ha exigido madurar en la gestión del tiempo, la planificación de tareas y la toma de decisiones arquitectónicas bajo restricciones técnicas reales.

Finalmente, al enmarcarse en una doble titulación, este primer proyecto me ha dejado una base sólida y una motivación renovada para abordar el reto inminente del despliegue en la nube. Considero que el resultado obtenido con SPIRITBLADE no solo cumple con las expectativas académicas, sino que sienta un precedente de gran valor para mi inminente inserción en el mundo profesional como ingeniero informático y del *software*.

5.2. Trabajos futuros

El ciclo de vida de SPIRITBLADE no concluye con la implementación de su lógica de negocio y su validación en un entorno local. Como continuación directa de esta memoria, el proyecto evolucionará en el marco de un segundo Trabajo de Fin de Grado.

La arquitectura *stateless* diseñada en este primer proyecto sienta las bases fundamentales para el **segundo TFG**, el cual se centrará exclusivamente en el **despliegue, la alta disponibilidad y la escalabilidad en la nube**. Las futuras líneas de trabajo que se abordarán de forma inminente incluyen:

1. **Contenerización productiva y orquestación:** Transición de un entorno Docker local a un clúster de **Kubernetes**, definiendo manifiestos para los despliegues (*Deployments*), servicios (*Services*) y balanceadores de carga (*Ingress Controller*).
2. **Integración y despliegue continuo (CI/CD):** Ampliación de los *pipelines* de GitHub Actions actuales para que, además de compilar y ejecutar pruebas automatizadas, se encarguen de construir las imágenes Docker, subirlas a un registro (*Container Registry*) y actualizar automáticamente los despliegues en el entorno de producción.
3. **Adopción de servicios cloud:** Despliegue de la infraestructura subyacente (bases de datos gestionadas, almacenamiento estático para el *frontend* y gestión de secretos) utilizando los servicios nativos de un proveedor en la nube público.
4. **Agregación de estadísticas agregadas:** Implementación de un sistema de análisis de datos a gran escala para generar estadísticas globales sobre el rendimiento de los jugadores y/o tendencias en las estrategias de *League of Legends*.

Bibliografía

[1] Oracle Corporation, "Java 21 documentation," https://docs.oracle.com/en/java/javase/21/, 2023.

[2] VMware, Inc., "Spring boot 3.0 release notes," https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-3.0-Release-Notes, 2022.

[3] Project Lombok Authors, "Project lombok: spicy adventures in java," https://projectlombok.org/, 2024.

[4] Microsoft Corporation, "Typescript: Typed javascript at any scale," https://www.typescriptlang.org/docs/, 2024.

[5] Google LLC, "Angular documentation," https://angular.dev/, 2024.

[6] Oracle Corporation, "Mysql 8.0 reference manual," https://dev.mysql.com/doc/refman/8.0/en/, 2024.

[7] Auth0 by Okta, "Json web token introduction," https://jwt.io/introduction, 2024.

[8] Riot Games, Inc., "Riot developer portal - api documentation," https://developer.riotgames.com/, 2024.

[9] Google DeepMind, "Gemini api documentation," https://ai.google.dev/docs, 2024.

[10] The Git Community, "Git documentation," https://git-scm.com/doc, 2024.

[11] The Apache Software Foundation, "Apache maven project," https://maven.apache.org/, 2024.

[12] npm, Inc., "npm docs," https://docs.npmjs.com/, 2024.

[13] SonarSource SA, "Sonarcloud documentation," https://docs.sonarcloud.io/, 2024.

[14] Docker Inc., "Docker documentation," https://docs.docker.com/, 2024.

[15] J. M. R. Silva, "Una caída en cloudflare afecta a millones de usuarios de ChatGPT, X y League of Legends durante horas: "Fallamos a nuestros clientes y a Internet en general"," https://www.elmundo.es/economia/2025/11/18/691c6939e85ece671b8b45a4.html, 2025, accedido el 22 de abril de 2026.

[16] Anthropic, "Prompt engineering interactive tutorial," Documentación oficial de Anthropic. https://docs.anthropic.com/claude/docs/prompt-engineering, 2024, accedido el 24 de abril de 2026.

[17] S. Willison, "The Claude 3 Opus system prompt," Blog de Simon Willison. https://simonwillison.net/2026/Apr/18/opus-system-prompt/, 2026, accedido el 25 de abril de 2026.

[18] M. Fowler, "The practical test pyramid," https://martinfowler.com/articles/practical-test-pyramid.html, 2018.

[19] Atlassian, "What is kanban?" https://www.atlassian.com/agile/kanban, 2024.

Apéndices

A

Funcionalidades detalladas

Este anexo desglosa de manera exhaustiva los requisitos funcionales implementados en la plataforma SPIRITBLADE, clasificados según los distintos niveles de acceso de los usuarios.

A.1. Módulo de usuario no registrado

El alcance para el usuario no registrado se centra en la consulta ágil de perfiles y estadísticas generales del videojuego:

- **Buscar invocadores:** Validación y consulta de un *Riot ID* para mostrar el perfil completo o un mensaje de error claro si no se encuentra el jugador (ver Figura A.1).
- **Ver perfil y rango:** Muestra el avatar, nivel, *Riot ID*, rango actual en el modo competitivo (*tier*/división/LP), tasa de victorias (*winrate*) global y las partidas totales disputadas (ver Figura A.2).
- **Estadísticas de campeones:** Acceso rápido al *top* 3 de campeones más jugados por el invocador, mostrando su nivel de maestría y los puntos totales acumulados con cada personaje (ver Figura A.3).
- **Búsquedas recientes:** La página de inicio lista de forma dinámica las 9 búsquedas más recientes realizadas por cualquier usuario en la plataforma (ver Figura A.4).
- **Imágenes reales:** Muestreo en tiempo real de imágenes oficiales de posiciones y campeones extraídas directamente de los servidores estáticos de Riot Games.

- **Detalles de partida:** Historial detallado de cada encuentro que incluye fecha, modo de juego, duración de la partida, KDA del jugador, campeones seleccionados, roles desempeñados y puntuación de visión (*Vision Score*) individual (ver Figura A.5).
- **Historial paginable:** Historial público de partidas con soporte para paginación avanzada, permitiendo navegar por encuentros pasados sin saturar el rendimiento del navegador (ver Figura A.6).

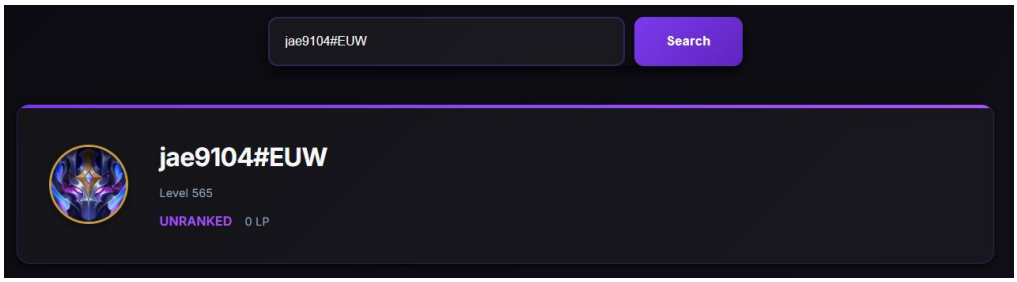


Figura A.1: Búsqueda de invocadores en la página principal.

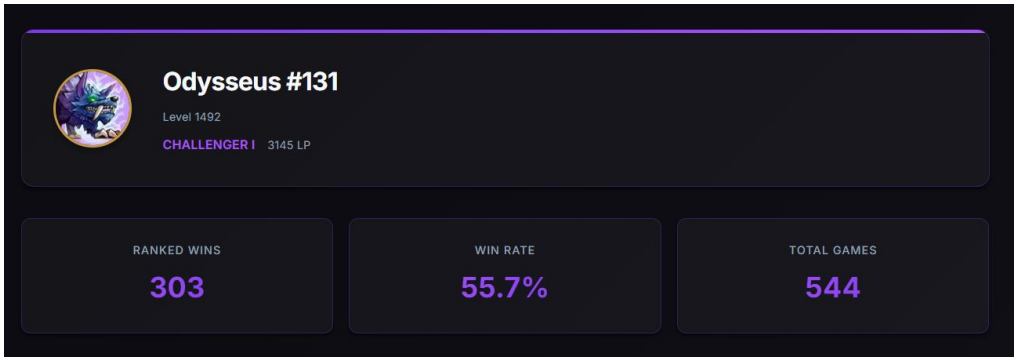


Figura A.2: Información general del perfil y rango de un invocador.

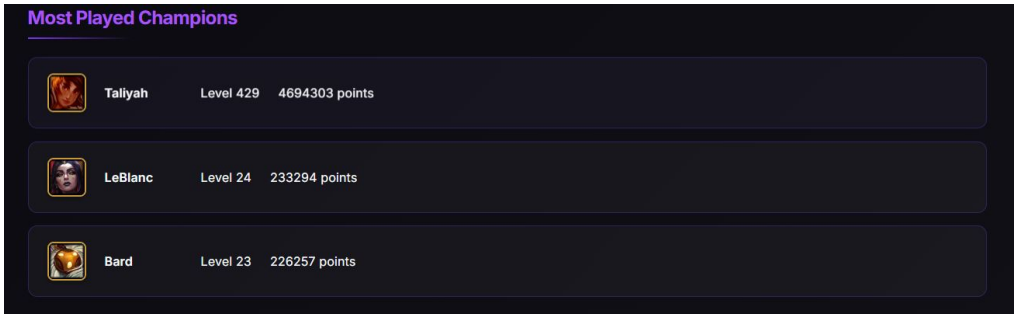


Figura A.3: Estadísticas de campeones más jugados.

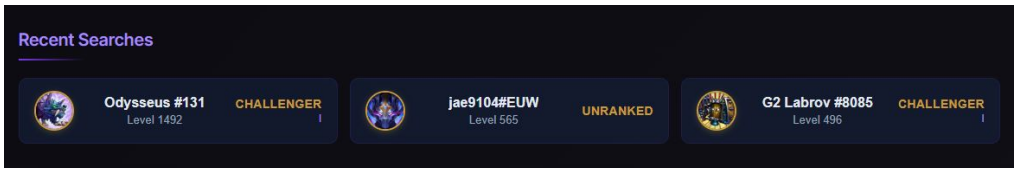


Figura A.4: Listado de búsquedas recientes de invocadores.

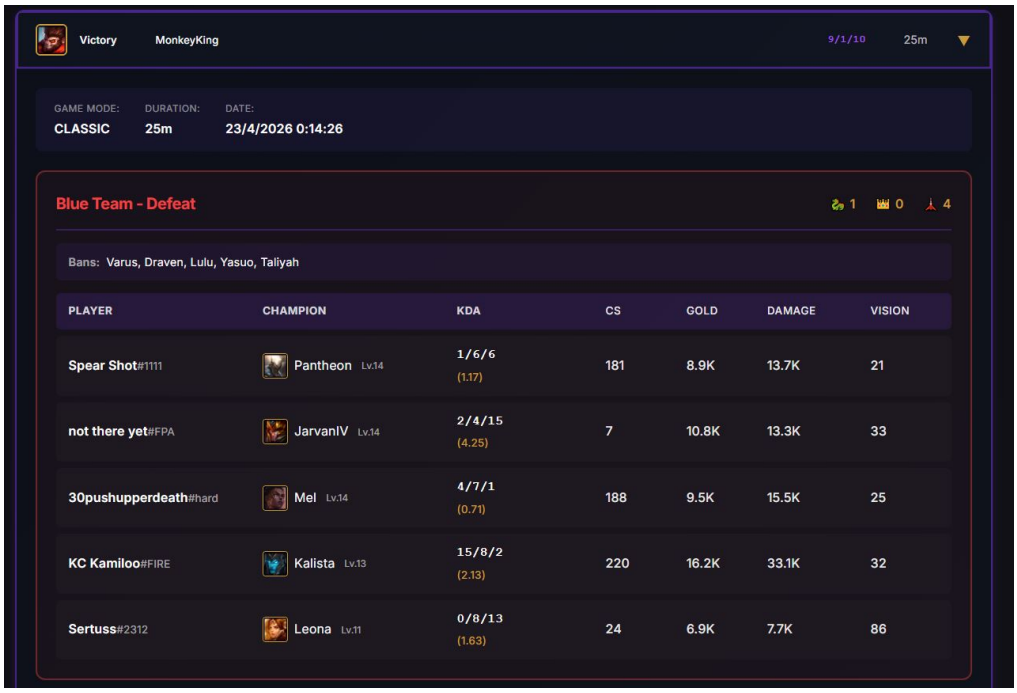


Figura A.5: Detalles específicos de una partida en el historial.

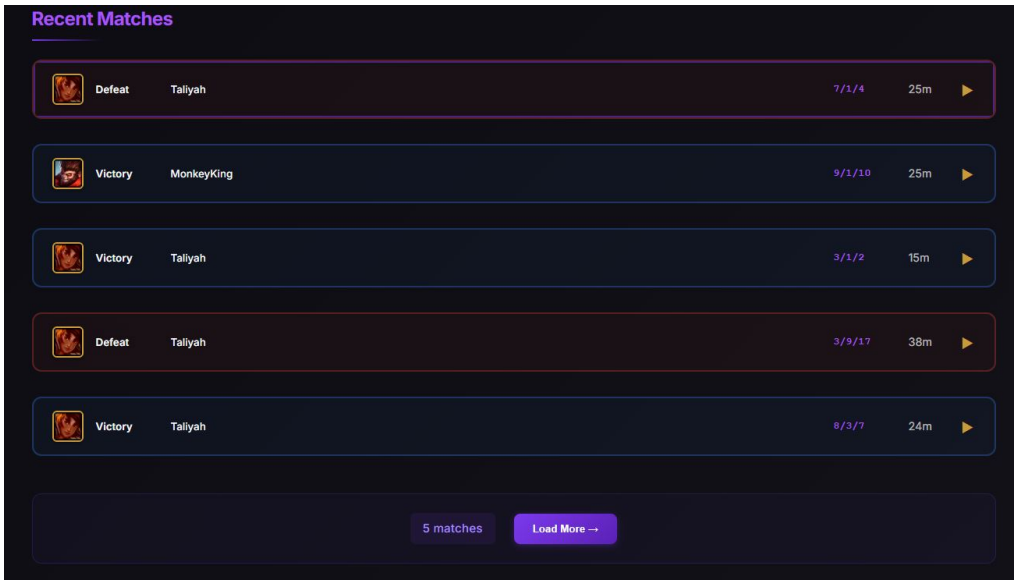


Figura A.6: Historial de partidas con paginación integrada.

A.2. Módulo de usuario registrado

Los usuarios autenticados acceden a capacidades de personalización y análisis avanzado mediante su panel de control privado:

- **Panel de control:** Panel (*dashboard*) unificado con información del perfil, estadísticas básicas y accesos directos a las acciones más comunes (ver Figura A.7).
- **Enlazar cuenta:** Capacidad para vincular una cuenta oficial de *League of Legends* usando el *Riot ID*, permitiendo el análisis automático de su perfil principal (ver Figura A.8).
- **Gestión de avatar:** Herramienta para subir y gestionar una foto de perfil personalizada almacenada en el servidor de objetos, MinIO (ver Figura A.9).
- **Editar perfil:** Funcionalidad para actualizar el correo electrónico y la contraseña, incluyendo una validación estricta de seguridad (ver Figura A.10).
- **Login/Registro:** Sistema completo de registro con correo electrónico y acceso seguro mediante el uso de JWT (ver Figura A.11).
- **Estadísticas del vinculado:** Mostrar el rango actual, los LP ganados recientemente, el rol principal desempeñado y el *Average Vision Score* del invocador vinculado (ver Figura A.12).
- **Sistema de favoritos:** Posibilidad de marcar invocadores de otros jugadores para su seguimiento rápido desde el *dashboard* sin necesidad de buscarles repetidamente (ver Figura A.13).
- **Análisis por IA:** Integración avanzada con la Inteligencia Artificial de Google Gemini para procesar el historial de partidas, ofreciendo un análisis del rendimiento, destacando fortalezas y proveyendo recomendaciones tácticas (ver Figura A.14).
- **Evolución de rango:** Visualización gráfica interactiva (mediante Chart.js) de la evolución temporal del rango (LP) en las colas clasificatorias (ver Figura A.15).

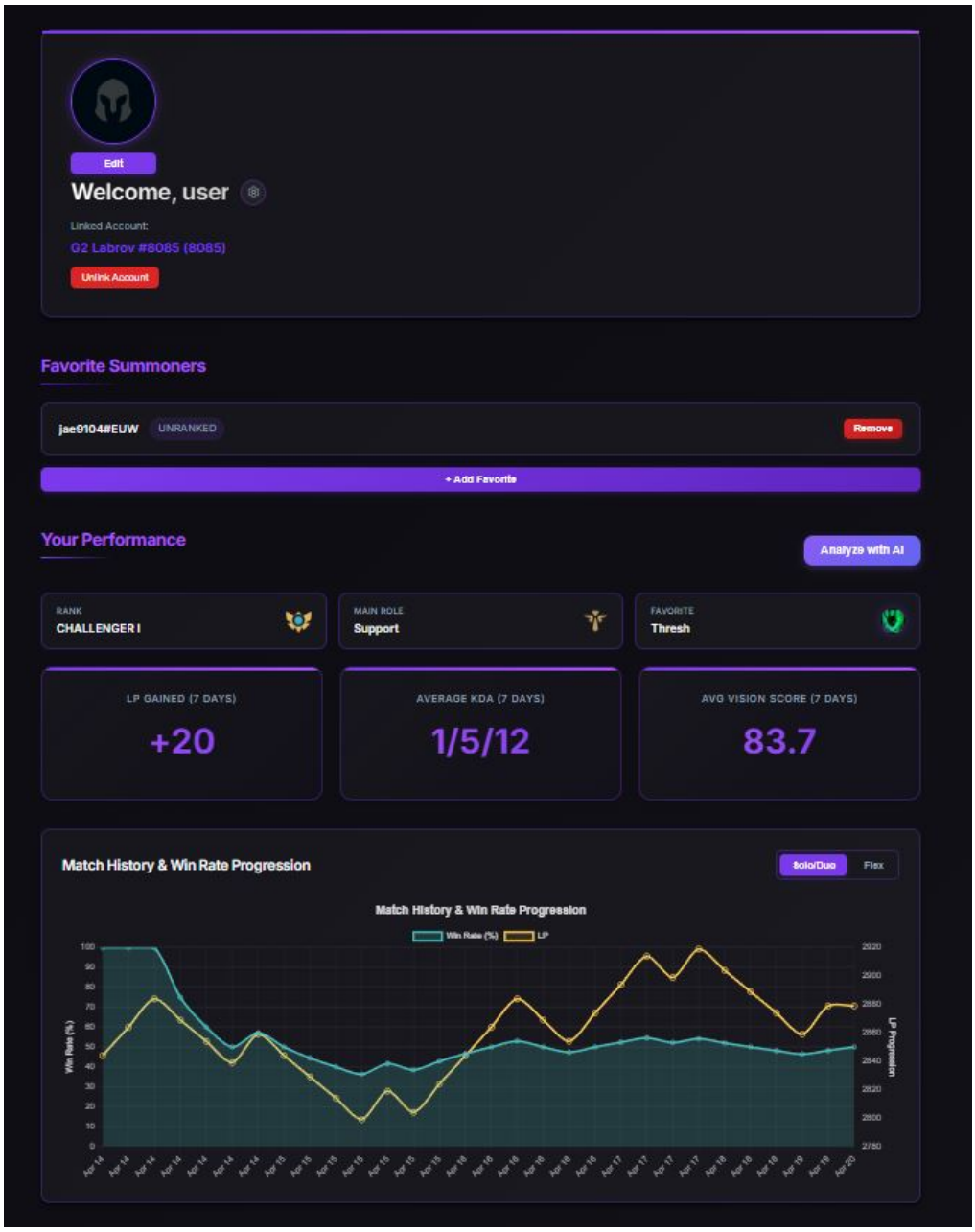


Figura A.7: Panel de control del usuario registrado.

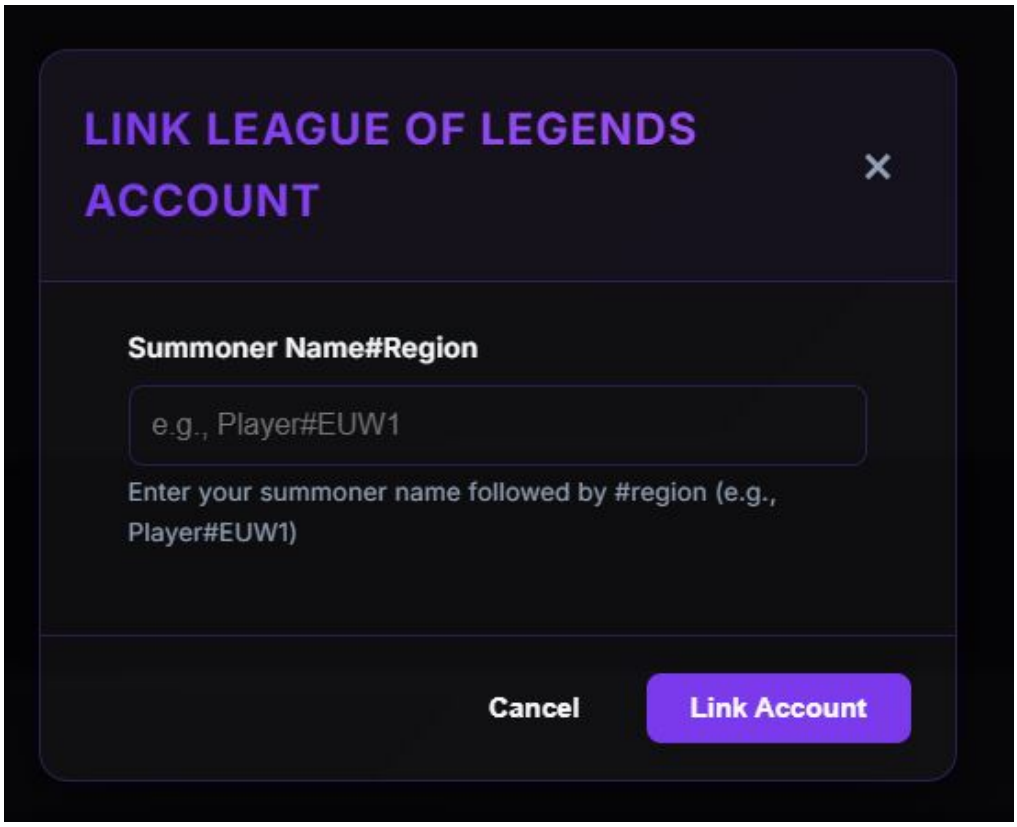


Figura A.8: Interfaz para enlazar la cuenta de Riot Games.

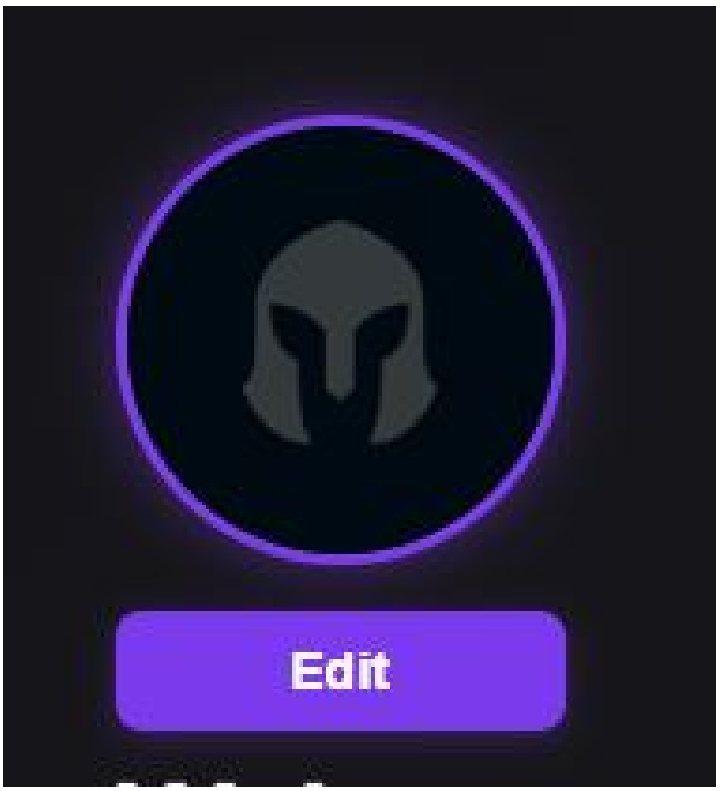


Figura A.9: Gestión de la foto de perfil del usuario.

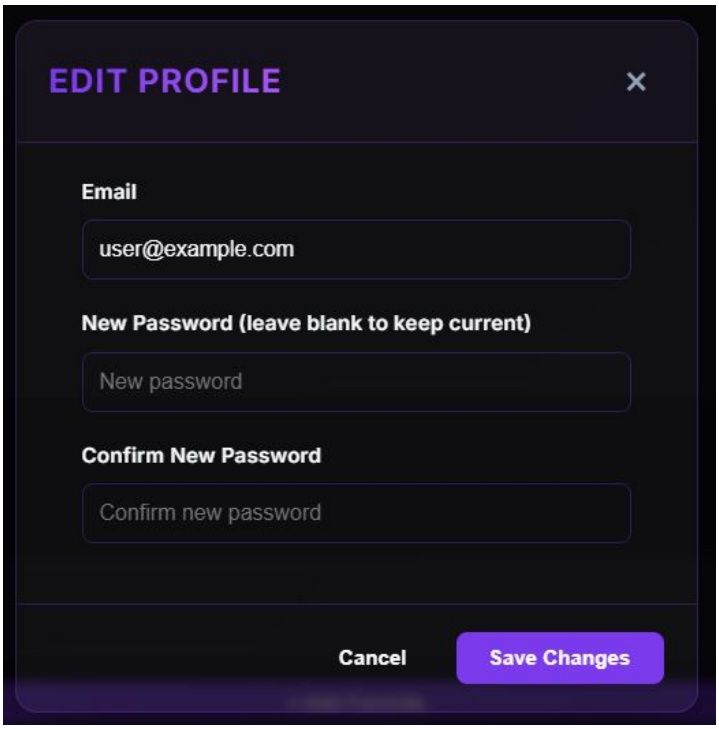


Figura A.10: Formulario de edición de datos de perfil.

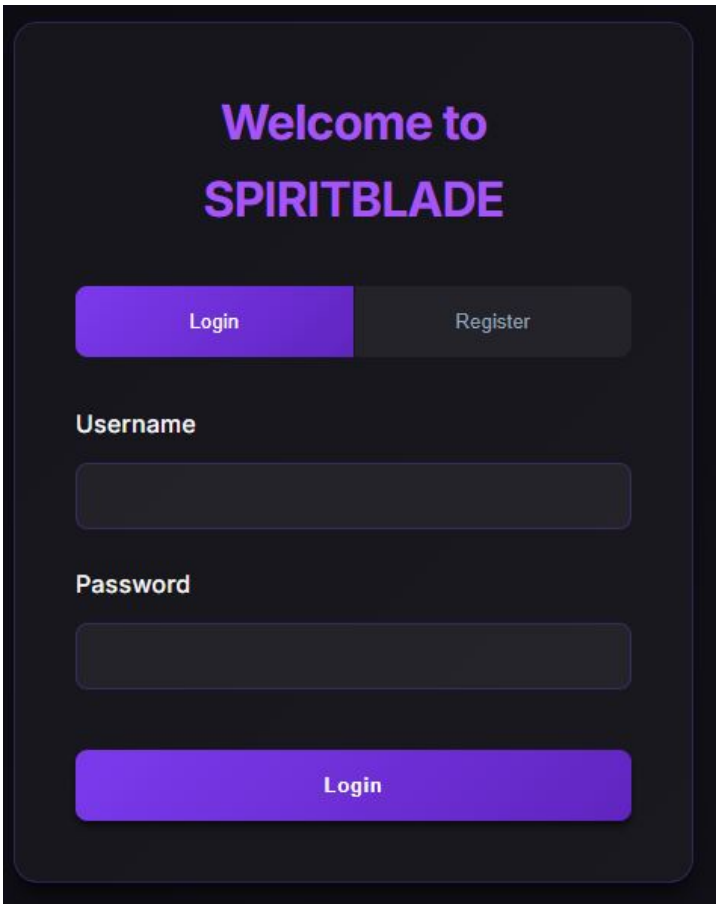


Figura A.11: Pantalla de inicio de sesión y registro en la plataforma.



Figura A.12: Estadísticas clave del invocador vinculado al usuario.

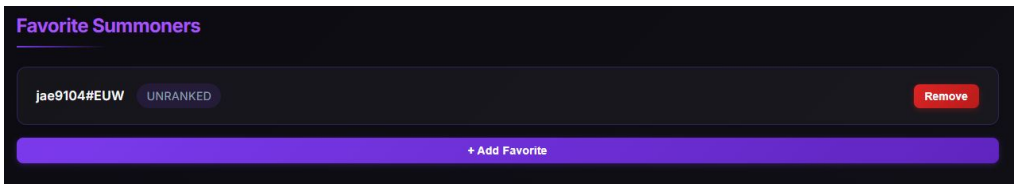


Figura A.13: Sección de invocadores favoritos para acceso rápido.

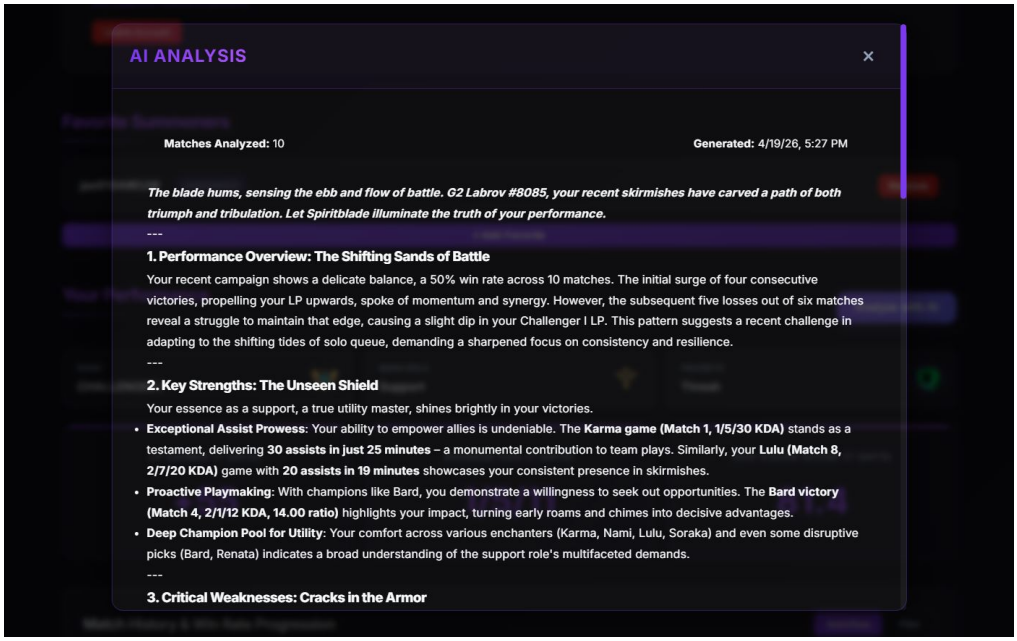


Figura A.14: Análisis de rendimiento generado por Inteligencia Artificial.

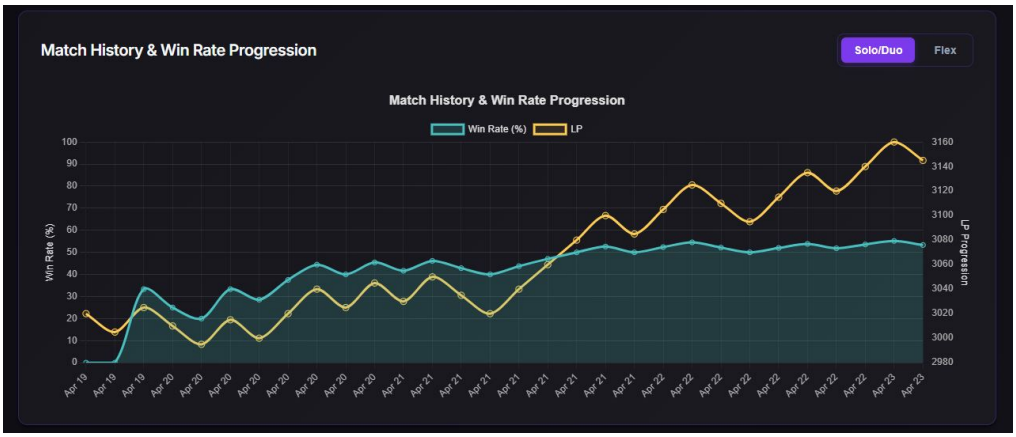


Figura A.15: Gráfico de evolución del rango (LP) a lo largo del tiempo.

A.3. Módulo de administración

Capacidades exclusivas orientadas a la gestión integral de la comunidad y la moderación del sistema:

- **Acceso al panel:** Ruta fuertemente protegida por el sistema de seguridad (*Spring Security*), accesible de manera exclusiva para cuentas con el rol de administrador (ver Figura A.16).
- **Listar usuarios:** Tabla de datos completa e interactiva con el ID interno, nombre, correo electrónico, roles asignados, estado actual de la cuenta y fecha de registro en la plataforma (ver Figura A.17).
- **Habilitar/Deshabilitar:** Control del acceso de los usuarios a la plataforma, permitiendo bloquear cuentas maliciosas de forma instantánea.
- **Eliminar usuarios:** Borrado permanente y en cascada de cuentas de usuario, requiriendo confirmación previa para evitar accidentes.
- **Editar roles:** Capacidad para promover usuarios estándar a administradores o revocar sus permisos.
- **Búsqueda avanzada:** Herramienta de filtrado combinado que permite buscar usuarios por nombre, correo electrónico, rol y estado, incluyendo paginación de resultados (ver Figura A.18).
- **Crear usuarios:** Formulario administrativo para dar de alta manualmente nuevas cuentas preconfigurando sus roles específicos.

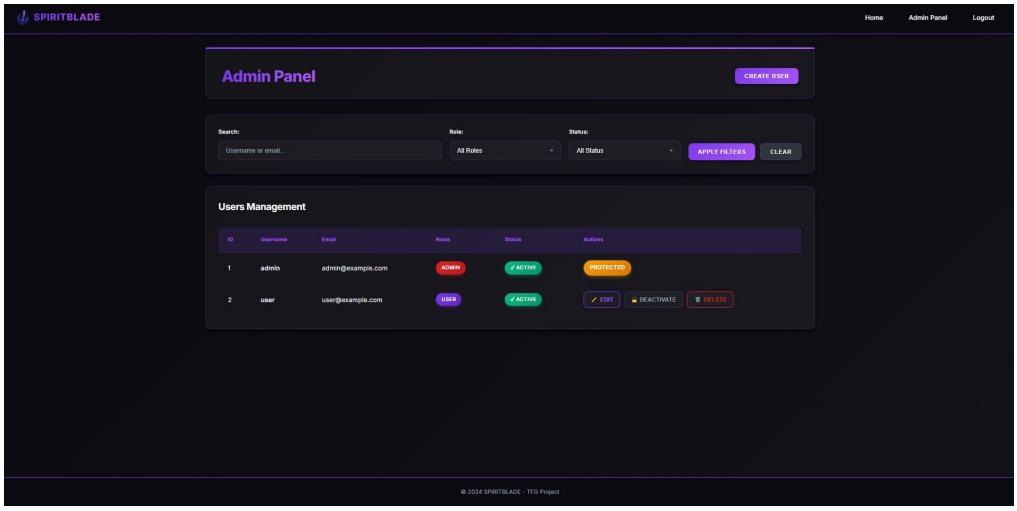


Figura A.16: Vista general del panel de administración.

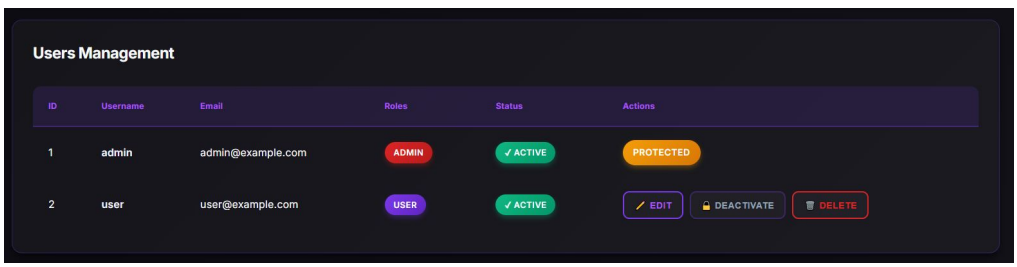


Figura A.17: Tabla interactiva con el listado de usuarios del sistema.

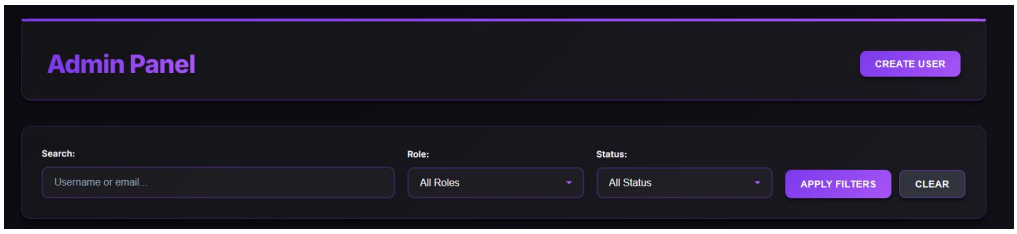


Figura A.18: Filtros de búsqueda avanzada en el panel de administración.

B

Instrucciones para ejecutar la aplicación

Este anexo detalla de manera estructurada el procedimiento necesario para el despliegue y la ejecución de la plataforma SPIRITBLADE en un entorno local utilizando contenedores. Siguiendo estos pasos, se garantiza que el sistema arranque en un entorno controlado y reproducible.

B.1. Requisitos previos

Para garantizar que la infraestructura de la aplicación se pueda levantar de forma contenerizada sin conflictos, el sistema anfitrión debe cumplir con los siguientes requisitos:

- **Docker Desktop:** Debe estar instalado y en ejecución activa. En sistemas operativos Windows, es imprescindible contar con el subsistema Subsistema de Windows para Linux (**WSL2**, del inglés *Windows Subsystem for Linux*) habilitado y correctamente integrado con Docker para garantizar el rendimiento de los contenedores.
- **Claves de API (*API Keys*):** Es necesario disponer de credenciales válidas generadas desde los portales para desarrolladores de Riot Games y Google Gemini. Sin estas claves, la aplicación no podrá realizar la recuperación de datos ni generar los resúmenes mediante inteligencia artificial.

B.2. Obtención del código fuente

El primer paso consiste en clonar el repositorio remoto en la máquina local. Abra su terminal y ejecute el siguiente comando para descargar la rama principal del proyecto:

```
git clone https://github.com/codeurjc-students/2025-SPIRITBLADE.git
cd 2025-SPIRITBLADE
```

B.3. Configuración del entorno (.env)

El proyecto emplea variables de entorno para manejar información sensible y parámetros de configuración sin exponerlos en el código fuente. Antes de iniciar los contenedores, es estrictamente necesario configurar este archivo:

1. Navegue mediante la terminal o el explorador de archivos al directorio `/docker`, el cual se encuentra ubicado en la raíz del proyecto.
2. Dentro de este directorio, localizará un archivo de plantilla denominado `.env.example`, que contiene las claves requeridas con valores vacíos.
3. Duplique dicho archivo en la misma carpeta y renombre la copia exactamente como `.env`.
4. Abra el archivo `.env` con cualquier editor de texto plano e introduzca sus credenciales reales. Deberá rellenar específicamente los campos correspondientes a las APIs externas (`RIOT_API_KEY`, `GOOGLE_AI_API_KEY`), el almacenamiento de objetos (`MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`), así como definir las contraseñas que utilizará la base de datos MySQL en su inicialización.

B.4. Ejecución del sistema

La plataforma SPIRITBLADE ha sido orquestada íntegramente mediante Docker Compose. Esta herramienta permite levantar simultáneamente todos los servicios necesarios: la base de datos relacional MySQL, el servidor de almacenamiento de objetos MinIO, el gestor de memoria caché Redis, y el contenedor principal de la aplicación que incluye Spring Boot y Angular.

Para poner en marcha el sistema, siga estos pasos:

1. Abra una terminal de comandos y asegúrese de estar situado en la carpeta `/docker` del proyecto (donde se ubica el archivo `docker-compose.yml`).
2. Ejecute el siguiente comando para construir (si es necesario) y levantar los contenedores en segundo plano:

```
docker-compose up -d
```

B.5. Acceso a la aplicación y credenciales

Una vez que el terminal indique que la aplicación se ha inicializado correctamente (generalmente con un mensaje de *Started SpiritbladeApplication*), la plataforma web estará completamente operativa.

Podrá acceder a la interfaz de usuario abriendo su navegador web e introduciendo la siguiente URL:

`https://localhost:443`

Nota sobre seguridad: Dado que el entorno local de desarrollo utiliza un certificado SSL autofirmado para forzar la navegación por HTTPS, su navegador mostrará una advertencia de seguridad indicando que la conexión no es privada. Es completamente seguro ignorar esta advertencia; deberá hacer clic en "Configuración avanzada" y seleccionar "Continuar a localhost (inseguro)" para acceder a la plataforma.

Para facilitar la evaluación inmediata de todas las funcionalidades, el sistema inicializa automáticamente la base de datos con dos perfiles de prueba:

- **Usuario regular:** Credenciales `user` / `pass`. (Asociado al rol `USER`, permite probar el panel de control y la IA).
- **Administrador:** Credenciales `admin` / `admin`. (Asociado al rol `ADMIN`, permite el acceso al módulo de administración y la moderación de la comunidad).

Nota sobre la gestión de credenciales: Es importante subrayar que la inclusión de estas credenciales por defecto responde exclusivamente a fines de demostración y agilidad en el entorno de desarrollo y evaluación académica. En un entorno de producción real, las contraseñas de usuarios y administradores nunca se expondrían ni se inicializarían de esta forma. Por el contrario, el resto de secretos críticos del proyecto (como las claves de las APIs de Riot Games o Google Gemini) se gestionan de forma segura mediante variables de entorno y archivos de configuración externos que nunca son rastreados por el sistema de control de versiones, garantizando la integridad y confidencialidad de la plataforma.

C

Ejecución y edición de código

Esta sección actúa como una guía técnica avanzada orientada a desarrolladores que deseen compilar, modificar el código fuente y levantar los módulos de SPIRITBLADE (*Frontend* y *Backend*) de forma independiente en su entorno local, sin recurrir a las imágenes preconstruidas de Docker.

C.1. Requisitos técnicos

Para poder intervenir en el desarrollo y compilar la aplicación nativamente, la estación de trabajo del desarrollador debe contar con las siguientes herramientas instaladas:

- **Kit de Desarrollo de Java (JDK, del inglés *Java Development Kit*) 21:** Es indispensable para ejecutar el *Backend* desarrollado en Spring Boot. Se recomienda la distribución de código abierto Eclipse Temurin.
- **Node.js y npm:** Se requiere la versión 18 o superior de Node.js, junto con su gestor de paquetes (npm), necesarios para instalar las dependencias del ecosistema de Angular 17.
- **Apache Maven:** Se requiere la versión 3.9 o superior para la gestión de dependencias y compilación de Java, así como Git para el control de versiones.
- **Base de Datos MySQL 8.0:** Puede estar instanciada nativamente en el sistema operativo local o mediante un contenedor Docker aislado.

C.2. Obtención del código fuente

El primer paso consiste en clonar el repositorio remoto en la máquina local. Abra su terminal y ejecute el siguiente comando para descargar la rama principal del proyecto:

```
git clone https://github.com/codeurjc-students/2025-SPIRITBLADE.git
cd 2025-SPIRITBLADE
```

C.3. Lanzamiento de los módulos en desarrollo

Antes de iniciar los servidores locales, asegúrese de haber creado un esquema de base de datos vacío llamado `spiritblade_db` en su instancia de MySQL y de haber completado la configuración del archivo `.env` tal y como se detalla en el Anexo B.

C.3.1. Servidor (Backend)

Es importante destacar una particularidad del ecosistema Spring Boot: no carga de forma nativa los archivos `.env` del sistema de ficheros de forma automática. Para evitar errores críticos durante la secuencia de arranque (causados por variables de configuración nulas, como la URL de la base de datos o las claves de las APIs), se debe inyectar el contenido del fichero `.env` directamente en las variables de entorno del sistema antes de lanzar el servidor mediante Maven.

Para solucionar esta problemática y agilizar el ciclo de desarrollo local, se ha diseñado un *script* automatizado. Si su entorno de trabajo es **Windows**, deberá utilizar PowerShell:

```
cd backend
# Script que parsea el archivo .env, carga las variables y arranca el servidor
.\dotenvtosystemargs.ps1
```

Si, por el contrario, opera bajo un sistema **Unix/Linux/macOS**, puede lograr el mismo efecto exportando las variables en la sesión de su terminal interactiva (Bash/Zsh):

```
cd backend
# 1. Lee el .env ignorando comentarios y exporta las variables
export $(grep -v '^#' ../docker/.env | xargs)
# 2. Lanzamiento del servidor usando el wrapper de Maven
./mvnw spring-boot:run
```

Una vez finalizado el arranque, la API REST quedará expuesta y operando en el puerto 443. Adicionalmente, los desarrolladores podrán explorar y probar interactivamente los *endpoints* a través de la interfaz de Swagger UI accediendo a `https://localhost/api/v1/swagger-ui.html`.

C.3.2. Cliente (Frontend)

El desarrollo de la interfaz se gestiona en un servidor independiente. Abra una nueva pestaña o ventana en su terminal, acceda al directorio del cliente, instale el árbol de dependencias de Node.js y ejecute el servidor de desarrollo de Angular (el cual ha sido configurado para integrar soporte de Capa de Conexión Segura (SSL, del inglés *Secure Sockets Layer*) local):

```
cd frontend
# 1. Descarga e instalacion de librerias de terceros declaradas en package.json
npm install
# 2. Compilacion en memoria y arranque del servidor con recarga activa
ng serve --ssl
```

La interfaz gráfica en modo desarrollo será accesible de inmediato mediante la ruta `https://localhost:4200`. Cualquier modificación en el código fuente de Angular provocará una recarga instantánea en el navegador, facilitando un ciclo de prueba rápido.

C.4. Compilación para producción (build)

Cuando el desarrollo ha concluido y se desea empaquetar el código fuente en artefactos finales optimizados (listos para su distribución o contenerización definitiva), deben ejecutarse las directivas de construcción para producción:

- **Generación del ejecutable JAR (Backend):** Este comando limpia compilaciones previas, empaqueta el código en un único archivo `.jar` y, opcionalmente, omite la ejecución de las pruebas unitarias para agilizar el proceso:

```
cd backend
.\mvnw.cmd clean package -DskipTests
```

- **Construcción de archivos estáticos minimizados (Frontend):** Este comando instruye a Angular para que procese el código TypeScript, elimine dependencias innecesarias, minimice los estilos y el código JavaScript, generando una carpeta `dist` lista para ser servida por un servidor web estático:

```
cd frontend
npm run build
```